

Resumen

Este Trabajo de Fin de Grado tiene el objetivo de diseñar y desarrollar una aplicación móvil multiplataforma para el control horario de los empleados de una empresa usando Flutter. La solución permite a la empresa gestionar el registro de la entrada y salida de los empleados con sus dispositivos móviles, además de otras funcionalidades extra. La solución también cumple con la normativa vigente sobre el control horario laboral.

El proyecto comenzó con una versión inicial hecha con Google Apps Script. Esa versión tenía limitaciones típicas de una aplicación web. Con la nueva aplicación, el proyecto pretende mejorar el rendimiento y la experiencia del usuario. Este incluye la autenticación con gestión de roles además del registro y consulta de fichajes, el cálculo automático de horas trabajadas y la petición de las vacaciones y gestión de nóminas, permitiendo que el administrador las suba y que cada trabajador pueda consultar la suya, así como un sistema de notificaciones internas en la app, que incluso permite fichar sin conexión a internet y, después, sincronizar los datos.

También, se pretende implementar roles específicos para la Inspección de Trabajo y la Representación Legal de los Trabajadores (RLT), anticipándose a la nueva ley de control horario digital que podría entrar en vigor a lo largo de este año.

Además, se usará Firebase como *backend* en la nube. Este nos da el almacenamiento, la sincronización en tiempo real y los mecanismos de autenticación seguros.



Resum

Este Treball de Fi de Grau té l'objectiu de dissenyar i desenrotllar una aplicació mòbil multiplataforma per al control horari dels empleats d'una empresa usant Flutter. La solució permet a l'empresa gestionar el registre de l'entrada i eixida dels empleats amb els seus dispositius mòbils, a més d'altres funcionalitats extra. La solució també complix amb la normativa vigent sobre el control horari laboral.

El projecte va començar amb una versió inicial feta amb Google Apps Script. Eixa versió tenia limitacions típiques d'una aplicació web. Amb la nova aplicació, el projecte pretén millorar el rendiment i l'experiència de l'usuari. Este inclou l'autenticació amb gestió de rols a més del registre i consulta de fitxatges, el càlcul automàtic d'hores treballades i la petició de les vacances i gestió de nòmines, permetent que l'administrador les puge i que cada treballador pugui consultar la seua, així com un sistema de notificacions internes en l'app, que fins i tot permet fitxar sense connexió a internet i, després, sincronitzar les dades.

També, es pretén implementar rols específics per a la Inspecció de Treball i la Representació Legal dels Treballadors (RLT), anticipant-se a la nova llei de control horari digital que podria entrar en vigor al llarg d'enguany.

A més, s'usarà Firebase com *backend* en el núvol. Est ens dona l'emmagatzematge, la sincronització en temps real i els mecanismes d'autenticació segurs.



Abstract

The main point of this Thesis is to design and develop a cross-platform mobile app for tracking employee work hours using Flutter. The solution enables the company to manage employee clock-in and clock-out records using their mobile devices, apart from other extra functionalities. The solution also complies with current regulations regarding work hour tracking.

The project began with an initial version built using Google Apps Script. That version had limitations typical of a web application. With the new application, the project aims to improve performance and the user experience. This includes role-based authentication, as well as clock-in and clock-out recording and viewing, automatic calculation of hours worked, vacation requests, and payroll management, allowing the administrator to upload payroll data and each employee to view their own, along with an internal notification system within the app. It even allows users to clock in without an internet connection and synchronize the data later.

Additionally, specific roles will be implemented for the Labor Inspectorate and the Workers' Legal Representative (RLT), in anticipation of the new digital time tracking law that could take effect later this year.

Additionally, Firebase will be used as the cloud backend. Which provides storage, real-time synchronization, and secure authentication mechanisms.

Agradecimientos

Quiero expresar mi agradecimiento a todas las personas que me han acompañado y apoyado durante el proceso de este Trabajo de Fin de Grado y la carrera en general.

En primer lugar, agradezco a mi tutor, Paco, por su orientación y ayuda durante el desarrollo del proyecto.

También quiero agradecer a mis compañeras/amigas de la universidad, Sandra y Oreto, por haber compartido conmigo estos años y por hacer mucho más amenas las clases y esta etapa en general, ¡a ver con quien juego yo en clase ahora! En especial, gracias a Oreto por contestarme a los cientos de mensajes y audios que empezaban con “holaa, duda...”, por aguantar todas mis dudas con el TFG y estar siempre dispuesta a ayudarme.

A mis tíos, gracias por confiar en mí para desarrollar esta aplicación y por creer que podría crear algo útil y funcional para la empresa. Gracias también por darme la oportunidad de aplicar mis conocimientos en un proyecto real. Y gracias también más concretamente a mi padre, por ser mi contacto directo con la empresa durante todo el desarrollo, por darme feedback constante y ayudarme a entender las necesidades reales que puede tener una empresa en una aplicación como esta.

Y, por supuesto, a mis padres, por su apoyo incondicional, su paciencia y por haber estado siempre a mi lado, incluso en mis momentos más dramáticos durante la carrera, especialmente durante el Erasmus, aguantando mis lloreras cada vez que suspendía, o creía haber suspendido, alguna asignatura. Gracias especialmente a mi madre, por soportar durante meses conversaciones casi exclusivas sobre el TFG y la aplicación y no habernos echado de casa ni a mi padre ni a mí después de meses con monotema en casa. Os quiero muchísimo y, mamá, prometo no volver a sacar el tema del TFG después de esto... aunque el de la aplicación va a ser más difícil evitarlo teniendo a papá usándola todos los días y a mí controlándola.

Finalmente, quiero dedicar unas palabras especiales a mi tía Concha, que desde siempre confió plenamente en mí. Para ella yo era poco menos que una hacker, experta en inglés. Siempre recurría a mí para cualquier cosa relacionada con la informática o el inglés, y con el tiempo entendí que lo realmente importante no era eso, sino la enorme confianza que tenía en mí y lo inteligente y capaz que me veía. Te quiero mucho y echo muchísimo de menos tus mensajes preguntándome cosas de informática o de inglés.

RESUMEN EJECUTIVO

La memoria del TFG del Grado en Tecnología Digital y Multimedia debe desarrollar en el texto los siguientes conceptos, debidamente justificados y discutidos, centrados en el ámbito de la tecnologías digitales y multimedia

CONCEPT (ABET)	CONCEPTO (traducción)	¿Cumple? (S/N)	¿Dónde? (páginas)
1. IDENTIFY:	1. IDENTIFICAR:		
1.1. Problem statement and opportunity	1.1. Planteamiento del problema y oportunidad	S	1-2
1.2. Constraints (standards, codes, needs, requirements & specifications)	1.2. Toma en consideración de los condicionantes (normas técnicas y regulación, necesidades, requisitos y especificaciones)	S	1-2 y 6-8
1.3. Setting of goals	1.3. Establecimiento de objetivos	S	3
2. FORMULATE:	2. FORMULAR:		
2.1. Creative solution generation (analysis)	2.1. Generación de soluciones creativas (análisis)	S	4-5 y 9-15
2.2. Evaluation of multiple solutions and decision-making (synthesis)	2.2. Evaluación de múltiples soluciones y toma de decisiones (síntesis)	S	4-5
3. SOLVE:	3. RESOLVER:		
3.1. Fulfilment of goals	3.1. Evaluación del cumplimiento de objetivos	S	33-39 y 41-42
3.2. Overall impact and significance (contributions and practical recommendations)	3.2. Evaluación del impacto global y alcance (contribuciones y recomendaciones prácticas)	S	39 y 41-42



Índice

Capítulo 1.	Introducción	1
1.1	Contexto.....	1
1.2	Motivación.....	1
1.3	Problema a resolver	1
1.4	Alcance del proyecto	1
Capítulo 2.	Objetivos	3
2.1	Objetivo general.....	3
2.2	Objetivos específicos	3
Capítulo 3.	Metodología	4
3.1	Metodología del desarrollo	4
3.2	Fases del proyecto.....	4
3.3	Herramientas utilizadas.....	4
Capítulo 4.	Análisis del sistema.....	6
4.1	Requisitos funcionales	6
4.1.1	Gestión de identidad y acceso	6
4.1.2	Gestión del registro de jornada.....	6
4.1.3	Gestión de Vacaciones	6
4.1.4	Gestión documental y nóminas	6
4.1.5	Sistema de notificaciones	6
4.1.6	Auditoría y cumplimiento normativo (RLT e Inspección).....	7
4.1.7	Resiliencia y modo offline.....	7
4.2	Requisitos no funcionales	7
4.3	Tipos de usuarios	7
Capítulo 5.	Diseño del sistema	9
5.1	Arquitectura del sistema	9
5.2	Diseño de la base de datos	9
5.3	Sistema de autenticación y control de acceso	10
5.4	Diseño de la interfaz	11
5.5	Gestión del modo offline	15
5.6	Despliegue web mediante Firebase Hosting.....	15
Capítulo 6.	Implementación.....	16
6.1	Entorno de desarrollo.....	16



6.2	Sistema de autenticación y gestión de usuarios	16
6.3	Gestión de roles y control de acceso	18
6.4	Gestión de empleados y asignación de roles.....	18
6.4.1	Arquitectura de datos: Users vs Employees	19
6.4.2	Administración y soporte técnico	19
6.5	Gestión de fichajes y horas trabajadas	19
6.5.1	Gestión de la persistencia y sincronización	20
6.5.2	Lógica de cálculo y clase WorkTime	20
6.5.3	Visualización y herramientas de supervisión	20
6.5.4	Exportación y cumplimiento normativo	21
6.6	Gestión de vacaciones.....	22
6.6.1	Lógica de cálculo y reglas de negocio.....	23
6.6.2	Herramientas de planificación administrativa	23
6.7	Gestión de incidencias y justificantes	25
6.7.1	Incidencias.....	25
6.7.2	Justificantes	26
6.7.3	Herramientas de supervisión y privacidad.....	26
6.8	Gestión de nóminas.....	26
6.9	Sistema de notificaciones.....	27
6.9.1	Alertas por perfil.....	27
6.9.2	Gestión del estado de lectura y persistencia	27
6.10	Visualización según rol (admin, worker, RLT, inspector).....	28
6.10.1	Configuración automática del entorno	28
6.11	Funcionamiento en modo offline	31
6.11.1	Gestión del estado de sincronización	31
6.11.2	Diferencias entre plataformas	32
Capítulo 7.	Tests y resultados	33
7.1	Tests unitarios	33
7.2	Tests funcionales.....	34
7.3	Tests de integración	38
7.4	Tests de usabilidad y compatibilidad.....	38
7.5	Resultados obtenidos	39
Capítulo 8.	Pliego de condiciones.....	40



8.1	Condiciones de funcionamiento.....	40
8.2	Estimación de costes.....	40
Capítulo 9.	Conclusiones	41
9.1	Conclusiones.....	41
9.2	Limitaciones	41
9.3	Trabajo futuro	42
Capítulo 10.	Bibliografía	43
Capítulo 11.	Anexos	44
11.1	Reglas de seguridad de Cloud Firestore.....	44
11.2	Reglas de seguridad de Firebase Storage.....	49
11.3	Índices de Cloud Firestore	50
11.4	Recursos del proyecto.....	51
11.4.1	Aplicación web desplegada.....	51
11.4.2	Repositorio de código fuente	51
11.4.3	Versión para Android (APK)	51
11.4.4	Credenciales de demostración.....	51

Índice de Figuras

Figura 1. Arquitectura de la base de datos en Firebase Firestore.....	10
Figura 2. Flujo de inicialización y autenticación.	11
Figura 3. Arquitectura funcional del panel trabajador.	12
Figura 4. Arquitectura funcional del panel administrador.....	14
Figura 5. Interfaces de registro e inicio de sesión (autenticación).	17
Figura 6. Interfaz y correo de recuperación de contraseña.....	18
Figura 7. Gestión de empleados y asignación de roles.....	19
Figura 8. Visualización diaria y mensual del historial de fichajes desde la vista del trabajador. 21	
Figura 9. Visualización diaria y mensual del historial de fichajes desde la vista del administrador.	21
Figura 10. Exportación en PDF del resumen mensual de horas trabajadas en perfil de trabajador y de administrador.....	22
Figura 11. Consulta del saldo y solicitudes de vacaciones, y panel de nueva solicitud.	23
Figura 12. Listado de peticiones y calendario mensual de vacaciones.	24
Figura 13. Cuadrante mensual de vacaciones.	24
Figura 14. PDF del cuadrante mensual de vacaciones.	25
Figura 15. Panel de incidencias y de nueva incidencia del trabajador, y del administrador.	25
Figura 16. Pantalla de justificantes de trabajador y empleado.	26
Figura 17. Pantalla de nóminas de trabajador y de administrador.	27
Figura 18. Pantalla de notificaciones y notificación flotante en pantalla.	28
Figura 19. Pantalla principal del trabajador y del administrador.	29
Figura 20. Panel de Inspección con y sin acceso, y vista restringida de un módulo.	29
Figura 21. Estructura funcional del panel inspector.....	30
Figura 22. Panel RLT, y vista restringida de un módulo.	30
Figura 23. Estructura funcional del panel RLT.....	31
Figura 24. Mensaje “fichaje guardado sin conexión” e icono de la nube en el registro.....	32
Figura 25. Reglas de acceso sobre las funciones generales.....	44
Figura 26. Reglas de acceso sobre el control de inspección y el usuario actual.	45
Figura 27. Reglas de acceso sobre la validación de notificaciones y los usuarios.	45
Figura 28. Reglas de acceso sobre los empleados y los fichajes.....	46
Figura 29. Reglas de acceso sobre las incidencias y vacaciones.....	46
Figura 30. Reglas de acceso sobre las vacaciones.....	47



Figura 31. Reglas de acceso sobre los justificantes y festivos locales.	47
Figura 32. Reglas de acceso sobre las nóminas y notificaciones.	48
Figura 33. Reglas de acceso sobre las notificaciones.....	48
Figura 34. Reglas generales y acceso a nóminas.....	49
Figura 35. Reglas de acceso a nóminas y justificantes.....	50
Figura 36. Índices en Cloud Firestore.	51



Índice de Tablas

Tabla 1. Gestión de errores en el proceso de autenticación	17
Tabla 2. Pruebas unitarias.	34
Tabla 3. Pruebas funcionales.....	38
Tabla 4. Pruebas de usabilidad y compatibilidad.	39

Capítulo 1. Introducción

1.1 Contexto

Lo que hace unos años era un trámite secundario en la administración de personal, hoy es obligatorio en España, cada empresa debe llevar un control riguroso de la jornada de trabajo de sus empleados gracias a la normativa vigente [1], lo que ha hecho que se evidencie una necesidad de digitalización.

Las pequeñas y medianas empresas son las que se han quedado atrás. Muchas siguen utilizando plantillas de Excel, o papel y firmas realizadas manualmente, y, esto no es posible en casos en los que el empleado no realiza su jornada en un centro físico, como en el caso del teletrabajo, por lo que estos métodos ya no siempre son eficaces.

1.2 Motivación

La idea de este trabajo nace de un problema real, transformar un sistema de gestión rígido en una herramienta ágil y moderna. El punto de partida fue una primera versión web que, aunque funcional, presentaba ciertas limitaciones en cuanto a rendimiento, experiencia de usuario y, sobre todo, una dependencia completa a la conexión a internet que no resultaba práctica.

Estas carencias se hicieron evidentes al observar ese problema en una empresa familiar dedicada al mantenimiento industrial, calefacción y fontanería. Al analizar su caso, se observó, que las aplicaciones genéricas del mercado no terminaban de encajar con ellos. Esta necesidad real ha permitido diseñar una herramienta a medida, centrada en digitalizar el control horario, mediante funciones que el propio gerente de la empresa ha propuesto y validado personalmente.

1.3 Problema a resolver

La clave de este proyecto es la falta de herramientas de control horario que sean sencillas de usar y a la vez, cumplan todos los requisitos de una empresa. En muchas empresas, los sistemas actuales fallan por varios motivos: son demasiado complejos para los trabajadores, fallan cuando no hay conexión a internet, o no son lo suficiente completos.

A esto se añaden retos operativos como la necesidad de gestionar distintos perfiles de usuario para empleados y administradores, o permitir un acceso rápido al historial de horas trabajadas. Por todo esto, la necesidad real no es solo el registro de jornada, sino crear un sistema que centralice toda la información de forma segura y accesible desde cualquier móvil.

1.4 Alcance del proyecto

Este trabajo se centra en el diseño y desarrollo de una aplicación móvil multiplataforma para gestionar la jornada laboral. El sistema permite al usuario autenticarse, registrar sus fichajes de entrada y salida, revisar su historial de fichajes, hacer incidencias en caso de olvidarse de fichar, subir justificantes, solicitar las vacaciones, consultar su nómina y en general, tener un control de su actividad diaria.

Para que la solución sea realmente útil en el día a día, la aplicación incluye:

- **Gestión de roles:** Control de Acceso Basado en Roles (RBAC, Rol-Based Access Control), diferenciando entre las funciones de un empleado y las de un administrador y, excepcionalmente, las de un inspector de trabajo o un representante legal de los trabajadores (RLT).
- **Sincronización híbrida:** Capacidad de registrar los datos de fichaje sin cobertura, y subirlos a la nube automáticamente al recuperar la conexión.



- **Almacenamiento de la nube:** Uso de un *backend* escalable para que la información esté siempre disponible.

Es importante aclarar, que el proyecto se entrega como un prototipo avanzado totalmente funcional, listo para ser probado en un entorno real.

Las normas de entrega del Trabajo Fin de Grado limitan la documentación oficial a esta memoria, por lo que para consultar el código fuente, probar la aplicación desplegada o descargar el instalable de Android, se han incluido todos los enlaces correspondientes en el anexo 11.4.

Capítulo 2. Objetivos

2.1 Objetivo general

El propósito central de este TFG es el diseño y desarrollo de una aplicación móvil multiplataforma que resuelva de forma eficiente la gestión del control horario. La solución busca ofrecer un sistema seguro y accesible desde cualquier dispositivo móvil, que no solo facilite el registro de la jornada laboral (entradas y salidas) y la consulta del historial horario, además de otras funcionalidades, sino que asegure el cumplimiento de la normativa vigente [1], de manera intuitiva para el trabajador.

2.2 Objetivos específicos

Para alcanzar la meta principal, el desarrollo se divide en los siguientes objetivos técnicos:

- **Diseño centrado en el usuario (UX/UI):** Implementación de una interfaz basada en los principios de *User Interface* (UI) y *User Experience* (UX). El objetivo de integrar ambas disciplinas es garantizar que la aplicación resulte intuitiva y accesible. La UX se encarga de que el flujo de navegación sea lógico y útil para el trabajador en su día a día (especialmente para aquellos menos familiarizados con la tecnología), y la UI asegura un entorno visual limpio y claro que facilite la interacción rápida y sin errores.
- **Control de acceso y perfiles:** Es fundamental la implementación de un sistema de *login* seguro. Este separará las funciones según el usuario: el empleado tendrá sus herramientas de fichaje y consulta, mientras que el administrador contará con un panel de gestión para supervisar al equipo y validar registros. Los usuarios para la Inspección de Trabajo y la RLT tendrán permisos de solo lectura y con un acceso restringido a la información.
- **Núcleo de la Aplicación:** Desarrollo de una lógica central que permita registrar las entradas y salidas, asegurando que los datos se guarden al instante y sin errores.
- **Procesamiento de jornadas y horas:** Más allá de guardar datos, la app debe ser capaz de calcular automáticamente el total de horas trabajadas, incluidas las horas extra. La idea es estructurar el historial para que el usuario pueda revisar su actividad de un vistazo, sin cálculos manuales.
- **Módulo de incidencias y ausencias:** Incorporación de herramientas para que el trabajador pueda reportar imprevistos (como olvidar fichar en algún momento), o subir justificantes directamente desde el móvil.
- **Gestión de nóminas:** Se habilitará un apartado específico donde el administrador pueda subir las nóminas de los trabajadores, permitiendo que cada empleado pueda consultar y visualizar de forma segura exclusivamente la suya, centralizando así toda la documentación.
- **Módulo de Vacaciones:** Diseño de un sistema de solicitudes donde el empleado pueda gestionar sus días de vacaciones desde la propia aplicación. Esto incluye un panel de administración para que el administrador pueda revisar, aprobar o denegar las peticiones, agilizando la comunicación sin necesidad de correos o formularios físicos.
- **Sistema de notificaciones internas:** Implementación de una funcionalidad de avisos dentro de la aplicación para agilizar la comunicación bidireccional entre el administrador y los trabajadores.
- **Arquitectura y Resiliencia:** Configuración de una base de datos en la nube que asegure que la información sea escalable y esté siempre disponible. A esto se le suma el permitir el uso *offline*, logrando que los datos se sincronicen automáticamente cuando el dispositivo recupere la conexión.
- **Optimización respecto a precedentes:** Superación de las limitaciones de rendimiento y fluidez detectadas en versiones web previas, aprovechando las capacidades nativas que ofrecen los *frameworks* móviles actuales.

Capítulo 3. Metodología

3.1 Metodología del desarrollo

Para asegurar que la aplicación respondiera realmente a lo que la empresa necesitaba, se ha descartado un diseño básico inicial y se ha optado por una metodología de desarrollo progresivo. El trabajo se ha organizado en iteraciones sucesivas: tras cada avance en el código, se ha realizado una fase de testeo y retroalimentación que ha servido de base para la siguiente mejora. Esto ha permitido que el sistema avance paso a paso, integrando soluciones técnicas a problemas reales detectados durante las pruebas.

Esta forma de trabajo ha resultado clave gracias a la colaboración directa con la gerencia de la empresa. El proceso ha seguido una lógica de validación constante: conforme se finalizaba una funcionalidad clave, como el sistema de fichajes, se ponía a prueba en el entorno real de la empresa. Este testeo continuo ha permitido que el proyecto creciera de forma natural, puesto que la interacción directa con la aplicación mostró necesidades que no se habían tenido en cuenta en la planificación inicial.

3.2 Fases del proyecto

El desarrollo del proyecto se ha organizado en siguientes fases de trabajo, asegurando que cada módulo técnico respondiera a las necesidades reales de la empresa:

- **Fase de análisis y diagnóstico:** En esta etapa inicial se han identificado los requisitos funcionales y técnicos requeridos por la empresa. Es clave el análisis de los problemas de la versión previa en Google Apps Script, cuya dependencia de la conectividad ha marcado las prioridades clave del nuevo sistema.
- **Fase de diseño de arquitectura:** Se ha definido la estructura global del *software*. Aquí se ha tomado la decisión de usar Flutter y Firebase [2], para soportar tanto el almacenamiento de datos como el de archivos físicos, el funcionamiento *offline*, y la compatibilidad con Android e iOS.
- **Fase de implementación técnica:** El desarrollo se ha realizado de forma modular, empezando por el acceso de usuarios y el motor de fichajes, para terminar con las funciones de gestión subida de archivos (justificantes y nóminas) y las notificaciones internas.
- **Fase de pruebas y validación real:** Se han realizado pruebas periódicas junto a la gerencia para confirmar que la aplicación sea intuitiva y resistente a errores en el día a día de la empresa.

3.3 Herramientas utilizadas

Para garantizar la escalabilidad y el rendimiento multiplataforma, se han seleccionado las siguientes herramientas:

- **Flutter y Dart:** Se ha optado por este SDK de Google para obtener un rendimiento nativo en Android e iOS con una única base de código. Su sistema de *widgets* ha permitido crear una interfaz intuitiva, algo fundamental para trabajadores que necesitan fichar rápidamente.
- **Ecosistema Firebase:** Se ha optado como solución de *Backend as a Service* (BaaS), actuando como la pieza central de la arquitectura del sistema.
 - o **Cloud Firestore:** Base de datos NoSQL elegida por su flexibilidad y capacidad para gestionar la sincronización *offline*, algo clave para el registro horario en zonas de baja cobertura.



- **Firebase Storage:** Servicio dedicado al almacenamiento de archivos, implementado específicamente para la gestión de documentos de ausencias de forma segura y eficiente.
- **Firebase Authentication:** Módulo utilizado para centralizar la seguridad de los accesos, permitiendo un control de identidad robusto y la gestión de sesiones de los empleados sin necesidad de desarrollar un servidor de autenticación desde cero.
- **Firebase Hosting:** Se ha empleado para desplegar la versión web de la plataforma, permitiendo el acceso rápido a través de navegadores sin necesidad de instalación previa.
- **Entorno de desarrollo:** El proyecto se ha realizado con Visual Studio Code, aprovechando sus extensiones específicas para el desarrollo con Dart [3]. Para asegurar el funcionamiento del *software*, las pruebas de la aplicación han combinado el uso de emuladores de Android Studio con pruebas en dispositivos físicos reales.

Capítulo 4. Análisis del sistema

4.1 Requisitos funcionales

Los siguientes puntos definen las capacidades operativas del sistema, estructuradas según las necesidades de negocio, y la experiencia de un usuario final.

4.1.1 *Gestión de identidad y acceso*

La plataforma integra un módulo de autenticación robusto para el control de accesos con los siguientes puntos:

- **Jerarquía de roles:** El sistema distingue perfiles de administrador, trabajador, representante legal de los trabajadores (RLT) e inspector de trabajo.
- **Control administrativo:** El administrador dispone de autorización completa para crear, modificar o eliminar cuentas, así como para asignar roles específicos a cada perfil.
- **Seguridad granular:** El acceso a las funcionalidades está condicionado por el rol asignado. Además, el administrador cuenta con una vista de "simulación" que le permite visualizar la interfaz desde la perspectiva de un trabajador.

4.1.2 *Gestión del registro de jornada*

La funcionalidad central de la aplicación permite un control preciso de la jornada laboral:

- **Fichajes:** Los trabajadores registran sus entradas y salidas, guardando automáticamente la fecha y hora.
- **Cómputo automático:** La aplicación calcula de forma automática el tiempo trabajado, identificando y separando las horas extra realizadas.
- **Auditoría:** Se habilita la consulta del historial completo de registros y la exportación de estos.

4.1.3 *Gestión de Vacaciones*

Este módulo permite la planificación y administración de las vacaciones:

- **Solicitudes:** Los trabajadores gestionan sus peticiones de días libres a través de la app.
- **Flujo administrativo:** El administrador recibe estas solicitudes para su validación (aprobación o rechazo), manteniendo al usuario informado mediante estados (pendiente, aprobado, rechazado).

4.1.4 *Gestión documental y nóminas*

El sistema integra herramientas orientadas a unificar la gestión administrativa, facilitando el acceso y control de la información:

- **Justificantes:** Los empleados pueden adjuntar documentos asociados a ausencias directamente desde el móvil, los cuales son almacenados y gestionados de forma segura.
- **Nóminas:** El administrador dispone de un espacio para subir las nóminas individuales. El sistema garantiza que cada trabajador pueda consultar exclusivamente su propia documentación.

4.1.5 *Sistema de notificaciones*

La comunicación bidireccional es clave para la fluidez del sistema:

- **Alertas:** El sistema informa al trabajador sobre cambios de estado en sus solicitudes (vacaciones, documentos, etc.).
- **Gestión de avisos:** El administrador es notificado ante cualquier pendiente que requiera su intervención.
- **Historial:** Se permite la consulta de notificaciones pasadas para mantener el rastro de la comunicación, también se permite la eliminación de estas para ahorrar espacio.

4.1.6 Auditoría y cumplimiento normativo (RLT e Inspección)

Adaptándose a la legislación vigente y a las futuras exigencias de control digital:

- **Roles de auditoría:** Se han diseñado perfiles específicos para la RLT y la Inspección de Trabajo, limitando sus permisos exclusivamente a la lectura y consulta de datos.
- **Acceso restringido:** Esta funcionalidad garantiza que dichos usuarios accedan únicamente a la información necesaria para la verificación legal, protegiendo la privacidad de los empleados y asegurando la integridad de los datos.

4.1.7 Resiliencia y modo offline

La aplicación garantiza la operatividad continua:

- **Persistencia:** Ante la ausencia de conexión, el sistema permite registrar los fichajes en modo local.
- **Sincronización:** Una vez restablecida la cobertura, los datos se sincronizan automáticamente con el servidor en la nube, asegurando que no se pierda información.

4.2 Requisitos no funcionales

Esta sección detalla las propiedades que definen el comportamiento y la robustez del sistema, asegurando su viabilidad operativa y la satisfacción del usuario final.

- **Seguridad:** La base de datos se fundamenta en protocolos de autenticación robustos. La información sensible se protege mediante acceso restringido con las reglas de Firebase [4], que se encuentran detalladas en el anexo 11.1, garantizando la integridad y confidencialidad en el almacenamiento en la nube.
- **Usabilidad:** Se ha optado por un diseño centrado en el usuario, priorizando la claridad y la accesibilidad. El objetivo es que cualquier empleado, independientemente de su competencia digital, pueda usar la aplicación sin problemas. Además, se ha implementado un diseño adaptativo que adecúa la interfaz a distintos tamaños de pantalla. Aunque la app está pensada para usarse principalmente en dispositivos móviles, se ha contemplado la posibilidad de usarla desde el ordenador, especialmente el administrador, a través de la versión web con Firebase Hosting [4].
- **Disponibilidad y resiliencia:** El sistema ha sido diseñado bajo un enfoque offline. Aunque se garantiza la operatividad con disponibilidad de red, la arquitectura permite el registro continuo de datos sin conexión, sincronizándose automáticamente en cuanto se restablece la cobertura.
- **Escalabilidad:** La infraestructura del *backend* está dimensionada para soportar un crecimiento progresivo, tanto en volumen de datos almacenados como en número de usuarios concurrentes, manteniendo los niveles de rendimiento óptimos ante el incremento de carga.

4.3 Tipos de usuarios

La aplicación implementa un modelo de Control de Acceso Basado en Roles (RBAC), asegurando que cada perfil interactúe exclusivamente con las herramientas y datos adecuados a su función.



- **Trabajador:** Usuario operativo con acceso a la gestión de su jornada laboral, consulta de nóminas, solicitud de vacaciones y recepción de notificaciones internas.
- **Administrador:** Perfil con privilegios totales de gestión. Supervisa la actividad de la plantilla, gestiona usuarios y roles, valida incidencias y vacaciones, consulta justificantes, gestiona nóminas y dispone de una vista de simulación de trabajador. También tiene acceso a recepción de notificaciones internas
- **Representante Legal de los Trabajadores (RLT):** Rol de auditoría con permisos de solo lectura, destinado a la consulta y supervisión del cumplimiento normativo en materia de control horario, garantizando la transparencia laboral.
- **Inspector:** Perfil técnico de supervisión legal. Dispone de un acceso restringido a solo lectura para verificar la veracidad de los registros y asegurar el cumplimiento de la normativa vigente [1].

Capítulo 5. Diseño del sistema

5.1 Arquitectura del sistema

Para este proyecto se ha optado por una arquitectura cliente-servidor basada en servicios en la nube. La aplicación, desarrollada en Flutter [5], actúa como el cliente donde el usuario interactúa con la interfaz. Por su parte, Firebase [4] hace de *backend*, proporcionando los servicios necesarios para la autenticación, el almacenamiento de datos y la sincronización.

En cuanto a la estructura de servicios, se ha optado por Firebase [4] con un modelo *Backend-as-a-Service* (BaaS). Esta elección ha sido para simplificar la arquitectura, ya que permite delegar la gestión del servidor y la escalabilidad a Google, eliminando la necesidad de configurar y mantener un servidor propio desde cero. Sin embargo, aunque la plataforma tecnológica es un BaaS, el sistema se ha implementado bajo una arquitectura *Single-Tenant*, lo que significa que la instancia se ha diseñado para servir a una única organización, por lo que para dar soporte a más empresas de forma independiente sería necesario replicar el proyecto.

5.2 Diseño de la base de datos

Para guardar toda la información de la aplicación, se ha optado por Cloud Firestore, es una base de datos NoSQL basada en documentos. Se ha elegido este sistema porque es muy flexible y permite organizar los datos en colecciones y documentos de manera sencilla, adaptándose perfectamente a las necesidades del proyecto. Se ha diseñado el modelo de datos pensando en las funcionalidades de la app y en los diferentes perfiles de usuario definidos.

La información se organiza en las siguientes colecciones principales:

- **users:** Colección base, donde se guarda la información del perfil (nombre e identificador), los roles asignados (administrador, trabajador, RLT o inspector) y los datos necesarios para la autenticación.
- **employees:** Aquí se separan los datos del usuario de los puramente laborales, como el departamento.
- **punches (fichajes):** Registra cada entrada y salida. Al guardar la fecha y hora junto con el identificador del usuario, se pueden calcular las horas trabajadas y detectar las horas extra de forma sencilla.
- **incidents:** Centraliza las incidencias que surgen durante la jornada, como olvidarse de fichar en algún momento.
- **vacation_requests:** Gestiona las vacaciones, desde que el trabajador lanza la solicitud hasta que el administrador la aprueba o rechaza.
- **local_holidays:** Esta colección sirve para tener en cuenta el calendario laboral y los festivos de cada ubicación, lo que es clave para el cálculo de jornadas.
- **absence_justifications:** Aquí es donde los trabajadores adjuntan los archivos o justificantes de sus ausencias.
- **payrolls (nóminas):** Espacio dedicado a las nóminas de cada empleado, garantizando que estén almacenadas de forma segura y asociadas a cada usuario.
- **notifications:** Controla el sistema de avisos de la app, registrando las notificaciones generadas para informar sobre cambios en solicitudes o avisos administrativos.

La figura 1 representa la estructura general de la base de datos utilizada en Firebase Firestore [4], incluyendo las colecciones principales, subcolecciones y relaciones entre ellas.

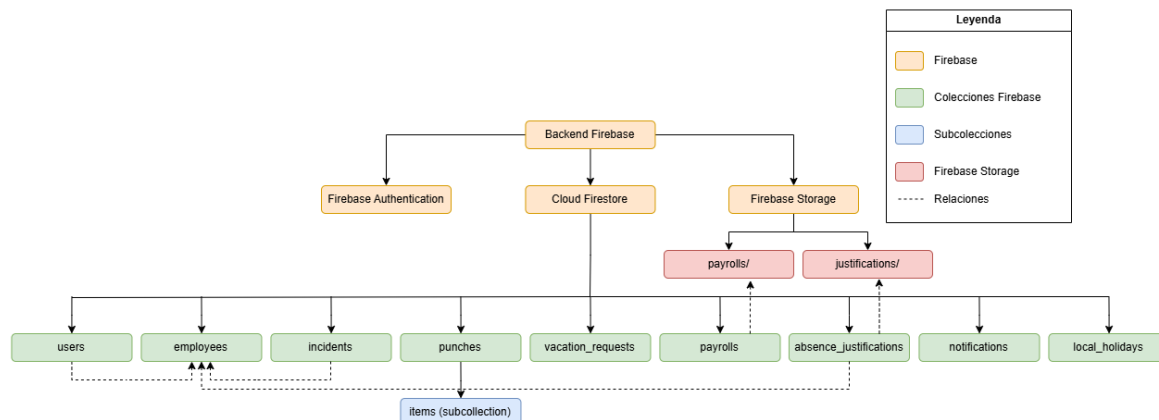


Figura 1. Arquitectura de la base de datos en Firebase Firestore.

Como norma general, cada documento incluye el identificador del usuario, lo que permite relacionar los distintos elementos (fichajes, nóminas, vacaciones) con la persona correcta.

5.3 Sistema de autenticación y control de acceso

Para gestionar el acceso, se ha integrado Firebase Authentication [4], utilizando el método de credenciales basadas en correo electrónico y contraseña.

Esta elección permite delegar la seguridad del almacenamiento de claves y la validación de identidad a un servicio especializado, garantizando que el proceso de *login* sea robusto desde el primer momento. Una gran ventaja de este módulo es la gestión automática de la persistencia de sesión, que se encarga de mantener la sesión activa, evitando que el usuario deba iniciar sesión cada vez que abre la aplicación

Sin embargo, para que el sistema sea realmente funcional, la autenticación se complementa con una base de datos personalizada en Cloud Firestore. El proceso funciona en dos pasos:

- **Identificación:** Firebase Auth [4] valida que el usuario existe y es quien dice ser.
- **Autorización:** Una vez autenticado el usuario, la app consulta automáticamente la colección users para extraer el perfil del trabajador.

En esta colección es donde está la lógica de Control de Acceso Basado en Roles (RBAC) basada en los cuatro perfiles definidos:

- Administrador
- Trabajador
- Representante Legal de los Trabajadores (RLT)
- Inspector

Finalmente, la interfaz de la aplicación se comporta de manera diferente según el rol obtenido, es decir, que el menú de navegación y las funcionalidades disponibles se filtran en tiempo real y cada perfil tiene acceso unas funcionalidades concretas predefinidas en las reglas de Firestore.

La figura 2 muestra la arquitectura general de acceso al sistema y el flujo de autenticación implementado mediante Firebase Authentication [4].

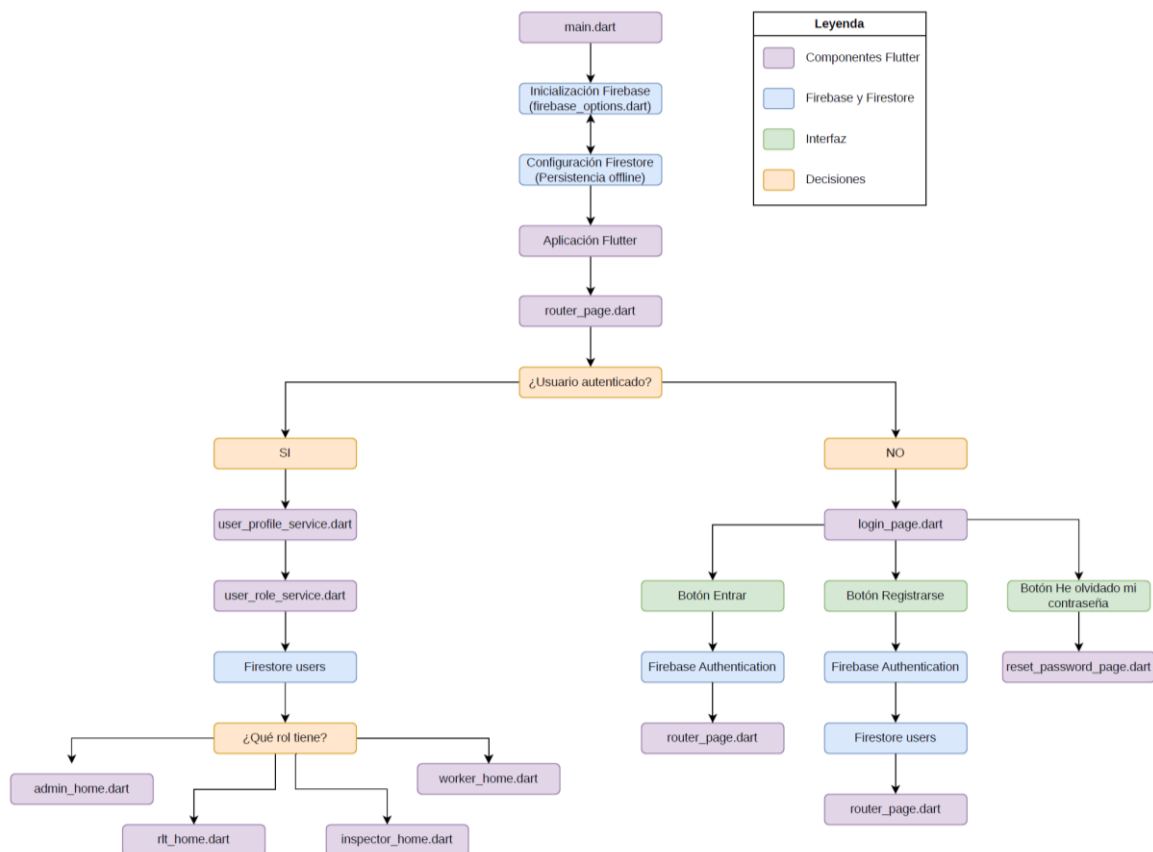


Figura 2. Flujo de inicialización y autenticación.

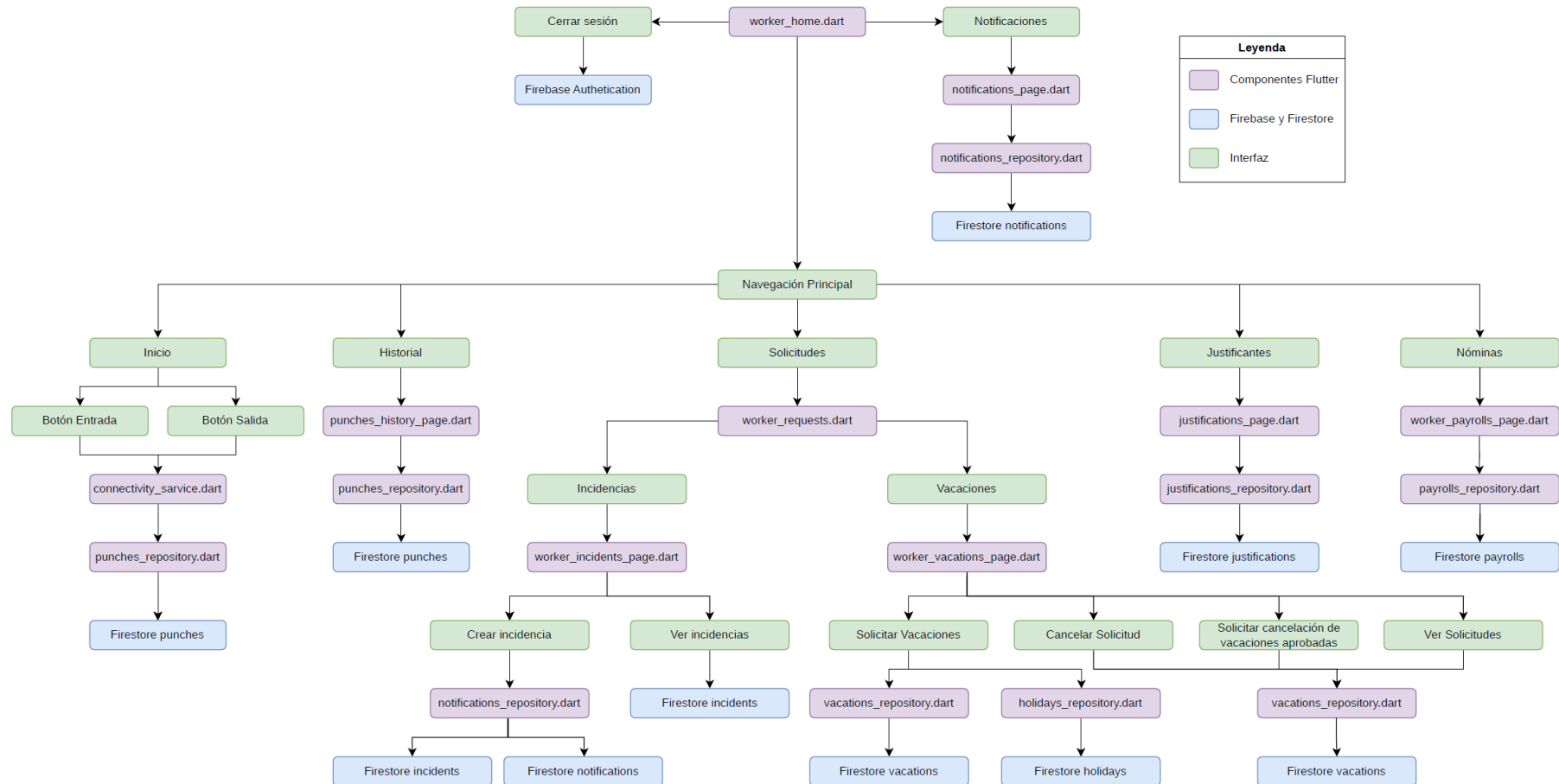
5.4 Diseño de la interfaz

Se ha desarrollado la interfaz utilizando Flutter [5], con una arquitectura de pantallas modulares que agrupan las distintas funcionalidades. El flujo de navegación está diseñado para ser sencillo y directo, permitiendo el acceso a las siguientes secciones:

- Pantalla de inicio de sesión
- Pantalla principal diferente para trabajador y para administrador:
 - Trabajador:
 - Pantalla de fichaje
 - Pantalla de consulta de historial de fichajes
 - Pantalla de solicitudes
 - Pantalla de incidencias
 - Pantalla de vacaciones
 - Pantalla de justificantes
 - Pantalla de notificaciones

La figura 3 muestra la estructura del panel del trabajador y las distintas funcionalidades disponibles.

Figura 3. Arquitectura funcional del panel trabajador.

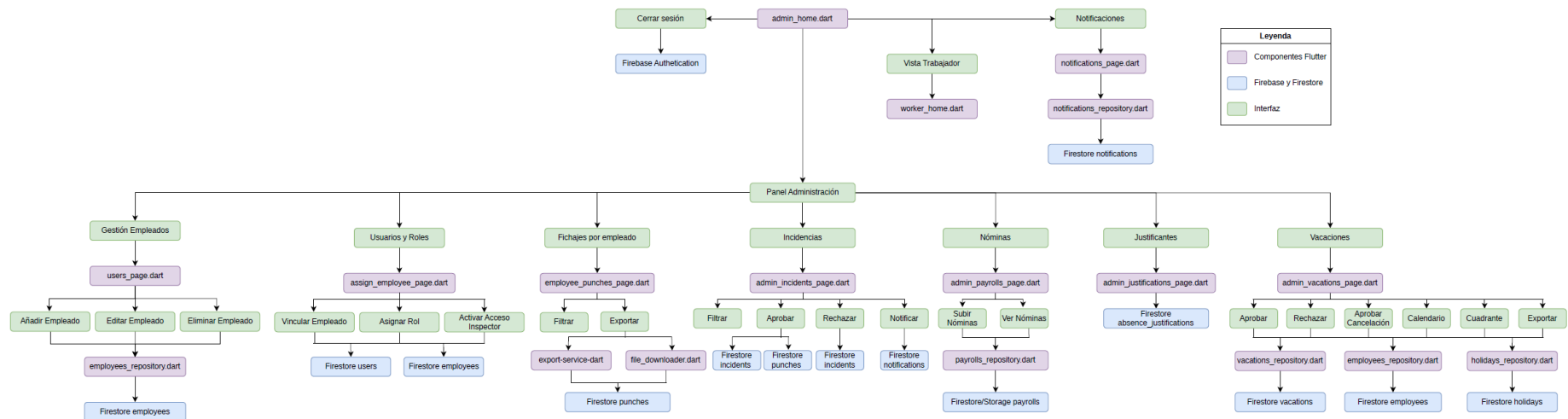




- Administrador:
 - Pantalla de panel de administración
 - Pantalla de gestión de empleados
 - Pantalla de asignación de empleados y roles
 - Pantalla de consulta de historial de fichajes por empleado
 - Pantalla de gestión de incidencias
 - Pantalla de gestión de vacaciones
 - Pantalla de gestión de justificantes
 - Pantalla de gestión de nóminas
 - Pantalla de notificaciones

La figura 4 representa la estructura del panel del administrador y las distintas funcionalidades disponibles.

Figura 4. Arquitectura funcional del panel administrador.



El objetivo principal al diseñar estas pantallas ha sido la simplicidad. Se ha buscado que el entorno sea visualmente claro para mejorar la experiencia de usuario y facilitar que cualquier persona pueda utilizar la aplicación de forma ágil, evitando problemas en la navegación entre las diferentes pantallas.

5.5 Gestión del modo offline

Uno de los puntos clave del proyecto es la capacidad de operar en modo *offline*. Para resolver los problemas de falta de conexión, se ha implementado un mecanismo que almacena la información localmente en el dispositivo cuando no hay red. Así, el usuario puede registrar su actividad sin interrupciones y, al volver a conectarse, la aplicación sincroniza los cambios con la base de datos de forma automática.

Este diseño mejora notablemente la disponibilidad del sistema, permitiendo que la aplicación sea útil incluso en condiciones de conectividad inestable.

5.6 Despliegue web mediante Firebase Hosting

Con el objetivo de mejorar la disponibilidad del sistema, se ha habilitado un acceso vía web mediante Firebase Hosting [4]. Esta decisión surge de la necesidad de ofrecer una alternativa a los usuarios de iOS, permitiendo el acceso a la plataforma desde el navegador funcionando como una solución de accesibilidad.

Sin embargo, esta versión web no es un sustituto completo de la app nativa. Al ejecutarse en el navegador, la aplicación depende de la conexión a internet, por lo tanto, la lógica del fichaje offline y la sincronización automática que se han desarrollado no funcionan en la versión web.

Capítulo 6. Implementación

6.1 Entorno de desarrollo

El proceso de desarrollo de la aplicación se ha centralizado en Visual Studio Code, configurando el ecosistema oficial de *plugins* para Flutter [5] y Dart [3]. Esta elección fue dada a que el entorno permita una integración fluida con herramientas de análisis de código.

Para la validación del software, se ha se han usado:

- **Emuladores de Android Studio:** Utilizados principalmente para pruebas rápidas de interfaz y ajustes de diseño.
- **Dispositivos físicos:** Fundamentalmente para probar la subida de archivos, las notificaciones, y, sobre todo, el comportamiento del modo offline, puesto que el emulador no siempre funcionaba correctamente.

El uso de *Hot Reload* de Flutter [5] ha marcado una gran diferencia en el desarrollo del código. Esta función permite que los cambios en los códigos se vean reflejados en la ejecución de la aplicación casi al instante, sin tener que reiniciarla. Esto ha permitido diseñar las interfaces mucho más rápido y corregir errores sobre la marcha, lo que ha agilizado mucho el proceso.

En cuanto a la arquitectura de *software*, se ha optado por una organización modular separando las distintas funcionalidades en pantallas, servicios y modelos de datos.

6.2 Sistema de autenticación y gestión de usuarios

La implementación técnica del acceso se ha estructurado siguiendo un diseño que prioriza la separación de responsabilidades. En lugar de gestionar el *login* directamente en la interfaz, se ha desarrollado un servicio de autenticación independiente que actúa como intermediario entre la aplicación y Firebase Authentication [4].

El flujo de ejecución implementado sigue una lógica de validación en dos niveles:

- **Validación de Identidad:** El servicio procesa las credenciales (email y contraseña) y gestiona las posibles excepciones del *backend* (como contraseñas incorrectas o usuarios inexistentes), proporcionando un control de errores robusto que informa al usuario en tiempo real, algunos de estas excepciones pueden verse en la tabla 1.
- **Extracción de Perfil:** Una vez confirmada la identidad, se dispara una consulta automática a la colección *users* en Cloud Firestore. Este paso es clave, ya que es donde se recupera el rol del usuario para configurar la navegación y habilitar o restringir funcionalidades antes de que el usuario llegue a la pantalla principal.

Código de error	Descripción	Mensaje mostrado al usuario
too-many-requests	Se han realizado demasiados intentos de acceso en poco tiempo	“Demasiados intentos. Inténtalo más tarde”
weak-password	La contraseña no cumple los requisitos mínimos de Firebase	“La contraseña debe tener al menos 6 caracteres”
invalid-email	El formato del correo electrónico no es válido	“El correo no es válido”
wrong-password	La contraseña introducida no coincide con la cuenta	“Contraseña incorrecta”

invalid-credential	Las credenciales introducidas no son válidas	“Correo o contraseña incorrectos”
email-already-in-use	Ya existe una cuenta con este correo electrónico	“Ya existe una cuenta con este correo electrónico”
too-many-requests	Se han realizado demasiados intentos de acceso en poco tiempo	“Demasiados intentos. Inténtalo más tarde”

Tabla 1. Gestión de errores en el proceso de autenticación

La figura 5 muestra las principales pantallas para el registro e inicio de sesión de los usuarios.

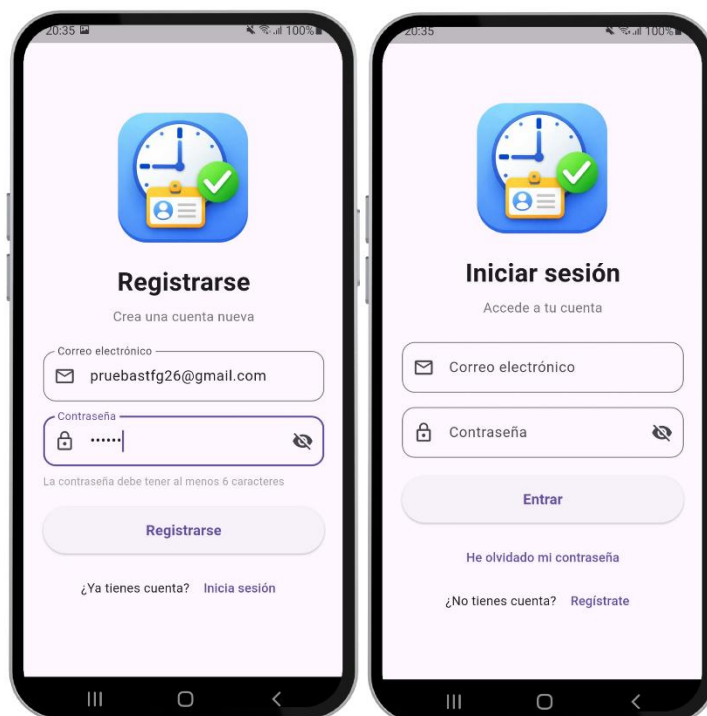


Figura 5. Interfaces de registro e inicio de sesión (autenticación).

Para mejorar la Experiencia de Usuario (UX), se han implementado indicadores de estado de carga (*loading states*) que evitan la incertidumbre durante la comunicación con el servidor. Además, se ha aprovechado la capacidad de persistencia nativa de Firebase [4], gracias a esto, el token de sesión se almacena de forma segura en el dispositivo, permitiendo que la aplicación reconozca al usuario en sucesivos accesos sin obligarle a pasar por la pantalla de *login*, a menos que cierre sesión explícitamente.

Este servicio no solo gestiona el inicio de sesión normal, sino que también integra un mecanismo de recuperación de credenciales. Cuando un usuario olvida su contraseña, puede iniciar un proceso de restablecimiento autónomo desde la propia aplicación. Al introducir su correo electrónico, el sistema dispara automáticamente un enlace de recuperación seguro, permitiendo que el empleado recupere el acceso sin que el administrador tenga que intervenir manualmente. Ambas funcionalidades pueden verse en la figura 6. No obstante, se ha detectado que estos mensajes pueden ser clasificados como spam, por ello, el sistema incluye un mensaje para que el usuario revise su bandeja de correo no deseado o spam.

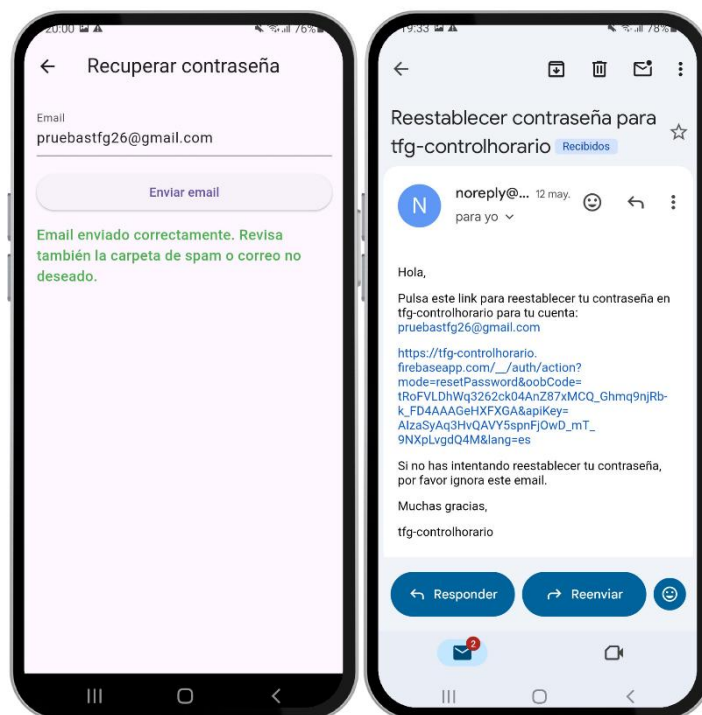


Figura 6. Interfaz y correo de recuperación de contraseña.

Este enfoque no solo facilita el mantenimiento del código, sino que permite que el sistema sea escalable, dejando la puerta abierta a integrar en el futuro otros métodos de acceso (como biometría) sin necesidad de modificar la lógica interna de la aplicación.

6.3 Gestión de roles y control de acceso

El sistema se ha diseñado bajo un modelo de Control de Acceso Basado en Roles (RBAC), cuyo objetivo es dividir las funcionalidades y el flujo de datos según el perfil del usuario. Una vez superada la fase de autenticación, la aplicación consulta el perfil almacenado en la colección users para establecer los permisos correspondientes en la navegación.

Para garantizar la integridad del sistema, esta lógica de control se divide en dos capas independientes y complementarias:

- **Capa de Interfaz (Frontend):** Se utiliza una visualización condicional para adaptar la interfaz en tiempo real según el perfil del usuario. La aplicación no solo oculta botones o menús, sino que restringe físicamente las rutas de navegación. Esto impide que un usuario pueda acceder a secciones no autorizadas, mejorando la experiencia de usuario al simplificar el entorno de trabajo según sus necesidades reales.
- **Capa de Persistencia (Backend):** Como medida de seguridad, se han programado Reglas de Seguridad en Cloud Firestore. Estas reglas actúan como un cortafuegos en el servidor que valida la identidad (request.auth) y los atributos del usuario antes de ejecutar cualquier operación de lectura o escritura. De este modo, se garantiza que los datos están protegidos contra intentos de acceso directo a la API de Firebase [4] que pretendan saltarse las restricciones de la interfaz.

Este enfoque de seguridad por capas asegura una arquitectura robusta, donde la privacidad de la información no depende únicamente de lo que el usuario ve, sino de una validación constante en el núcleo de la base de datos.

6.4 Gestión de empleados y asignación de roles

Para garantizar una organización de datos, se ha implementado un sistema de gestión que separa la identidad de acceso de la laboral. Este diseño permite diferenciar claramente quién accede al sistema y a qué información tiene acceso.

6.4.1 Arquitectura de datos: Users vs Employees

El sistema gestiona la información mediante dos colecciones independientes en Firestore, vinculadas de forma lógica:

- **Colección users:** Almacena las credenciales de acceso, el estado de la cuenta y el rol asignado dentro de la plataforma.
- **Colección employees:** Contiene el expediente laboral del trabajador.

Esta separación se resuelve mediante un mapeo a través de employeeId. Gracias a este identificador único, la aplicación es capaz de filtrar dinámicamente toda la actividad (fichajes, nóminas, solicitudes...) para que cada usuario interactúe únicamente con sus registros.

6.4.2 Administración y soporte técnico

El panel de control permite al administrador organizar la estructura del personal a través de tres funciones principales:

1. **Alta y vinculación:** Permite crear la ficha de empleado y asociarla a un usuario autenticado en el sistema.
2. **Gestión de privilegios:** Asigna los roles que determinan qué funcionalidades tendrá disponibles.

Las interfaces de la figura 7 permiten crear la ficha de los empleados y vincularlos con su cuenta, y asignar el rol que tendrán.

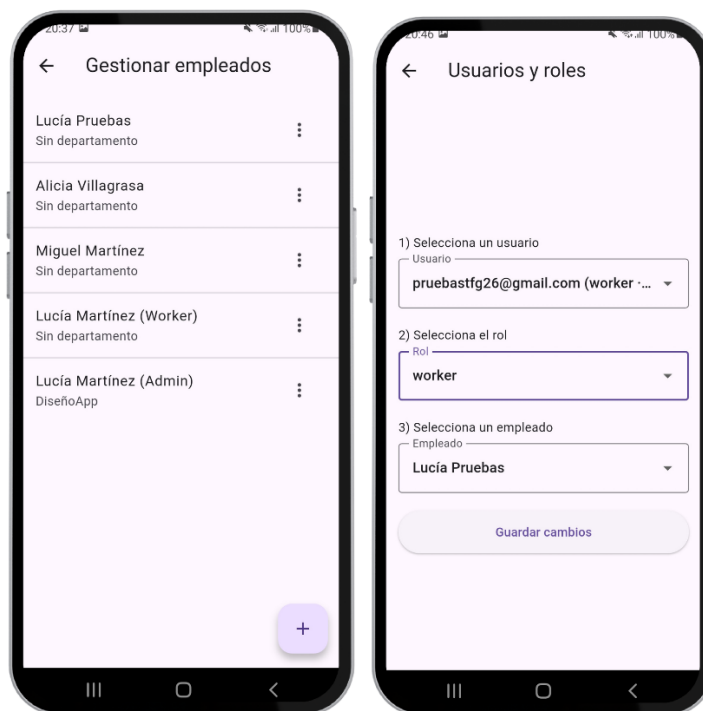


Figura 7. Gestión de empleados y asignación de roles.

6.5 Gestión de fichajes y horas trabajadas

El núcleo funcional de la aplicación es el sistema de fichajes, diseñado para que el trabajador pueda registrar sus entradas y salidas de forma intuitiva desde su pantalla principal. Para asegurar la fiabilidad de estos datos, la lógica se ha extraído de la interfaz y se ha centralizado en un repositorio especializado (PunchesRepository). Este componente no solo gestiona la persistencia en la base de datos, sino que actúa como primera capa de validación, comprobando la conectividad del dispositivo antes de cada operación.

6.5.1 Gestión de la persistencia y sincronización

Uno de los retos técnicos resueltos es la garantía de registro en condiciones de movilidad. Si el dispositivo pierde la conexión, la aplicación almacena el fichaje localmente y notifica al usuario que el dato se sincronizará automáticamente al recuperar el acceso a internet.

6.5.2 Lógica de cálculo y clase WorkTime

Para desacoplar el procesamiento matemático de la visualización, se ha creado la clase auxiliar WorkTime. Esta clase es la encargada de transformar los registros brutos en información útil para la empresa. El algoritmo funciona de la siguiente manera:

- **Emparejamiento cronológico:** Los fichajes se ordenan por fecha y se agrupan en parejas de entrada y salida pertenecientes al mismo día.
- **Segmentación de jornada:** Se ha definido un umbral de jornada ordinaria de 480 minutos (8 horas). El sistema calcula automáticamente la diferencia entre el tiempo total trabajado y este umbral para separar las horas ordinarias de las extraordinarias.
- **Filtros de integridad:** El cálculo se ha programado para ser robusto frente a errores de uso. El sistema ignora automáticamente salidas sin entrada previa, registros con duraciones negativas o jornadas de longitud ilógica, evitando que un olvido del trabajador al fichar rompa los cálculos.

6.5.3 Visualización y herramientas de supervisión

La consulta de estos datos se adapta según el perfil que accede al sistema:

- **Vista del Trabajador:** Permite filtrar por día o mes para revisar su historial y el acumulado de horas, tal como se muestra en la figura 8.
- **Vista de Administración:** Puede seleccionar a cualquier empleado y verificar no solo sus horas, sino también la localización de cada fichaje si el dispositivo tenía el GPS activo, como puede apreciarse en la figura 9.

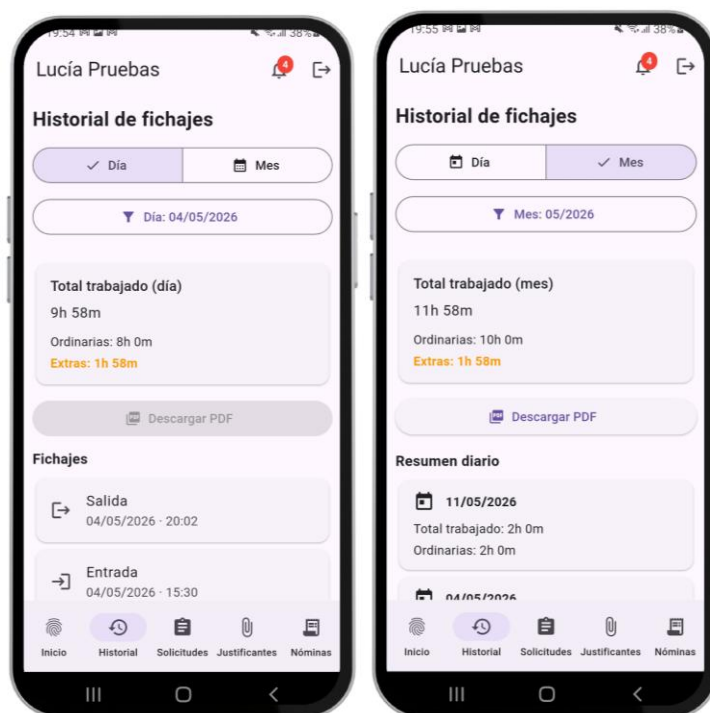


Figura 8. Visualización diaria y mensual del historial de fichajes desde la vista del trabajador.

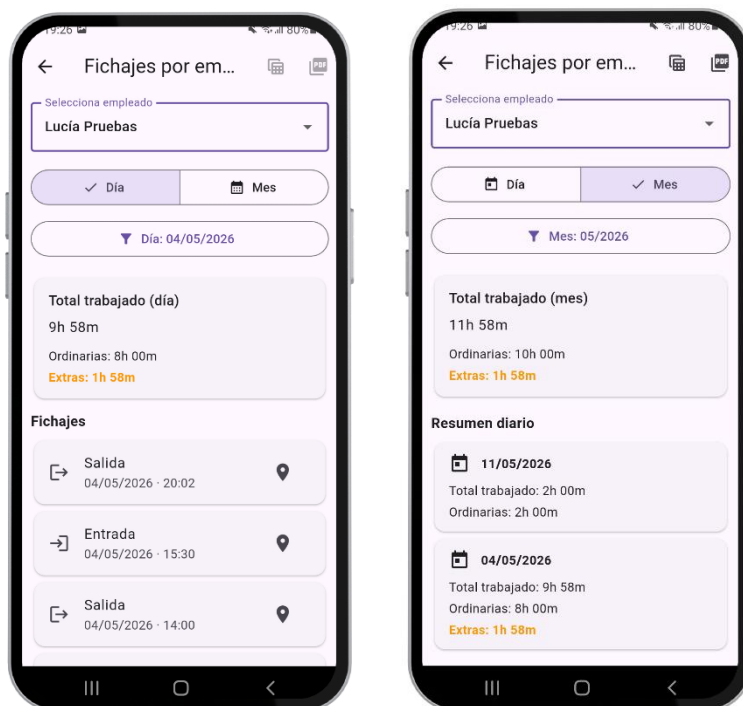


Figura 9. Visualización diaria y mensual del historial de fichajes desde la vista del administrador.

6.5.4 Exportación y cumplimiento normativo

Finalmente, para dar respuesta a las necesidades administrativas y legales, se ha integrado un módulo de exportación en formatos PDF y CSV.

La exportación en PDF está disponible tanto para el trabajador como para el administrador, garantizando que ambas partes dispongan de un documento no editable que certifique las horas realizadas mensuales. Por el contrario, la exportación en CSV se ha habilitado exclusivamente

para administrador, puesto que dicho formato es editable. Esta diferenciación se puede observar en la figura 10.

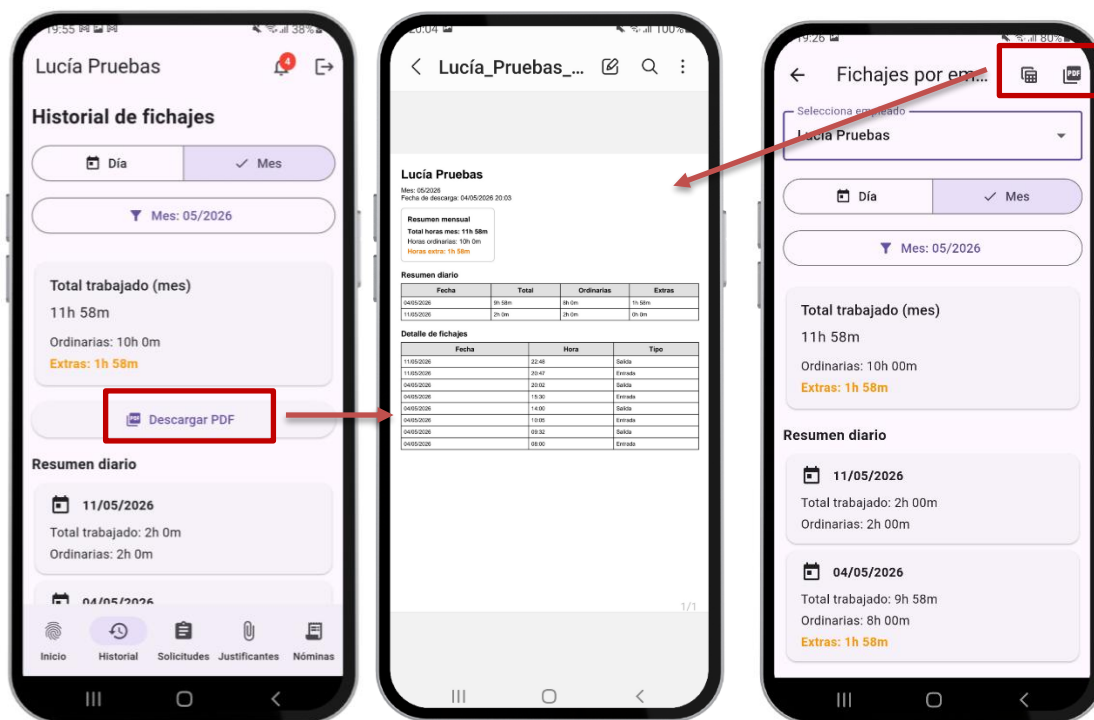


Figura 10. Exportación en PDF del resumen mensual de horas trabajadas en perfil de trabajador y de administrador.

6.6 Gestión de vacaciones

El sistema incorpora un módulo para la gestión de las vacaciones, permitiendo al trabajador supervisar su saldo anual, gestionar nuevas solicitudes y realizar un seguimiento de su estado, visible en la figura 11. El sistema se ha configurado con 22 días laborables anuales disponibles, sin contar fines de semana ni festivos.

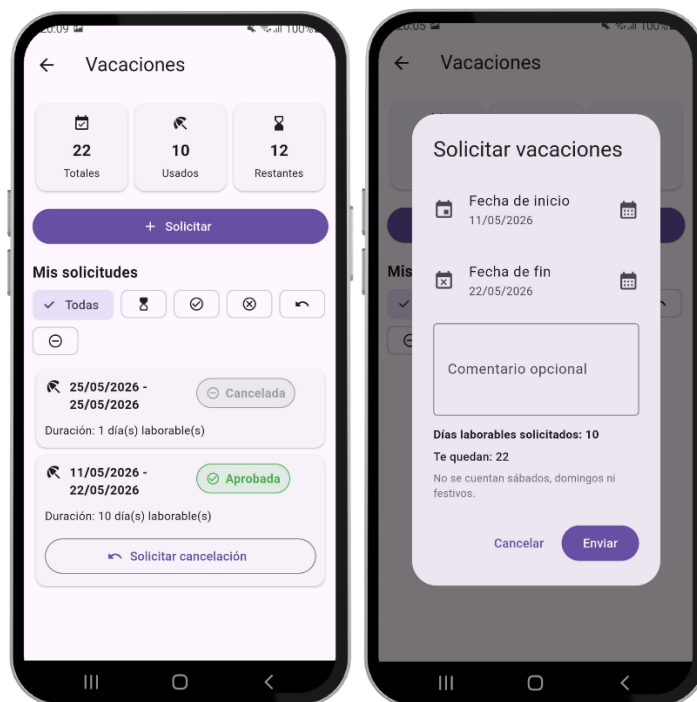


Figura 11. Consulta del saldo y solicitudes de vacaciones, y panel de nueva solicitud.

6.6.1 Lógica de cálculo y reglas de negocio

El sistema implementa un algoritmo que recorre el rango seleccionado por el empleado y filtra los días. Para garantizar que el saldo de vacaciones sea exacto, el sistema:

- **No cuenta días no laborables:** Excluye automáticamente sábados y domingos del cálculo.
- **Gestión de festivos:** Contrasta las fechas con una colección específica de festivos locales y nacionales almacenada en Firestore, evitando que estos días descuenten del saldo del trabajador.
- **Flujo de estados:** Las solicitudes se gestionan mediante un sistema de estados (*Pending, Approved, Rejected*). Se ha implementado una restricción de seguridad clave: mientras que una solicitud pendiente puede ser cancelada por el usuario, una ya aprobada requiere una petición de cancelación que debe ser validada por el administrador.

6.6.2 Herramientas de planificación administrativa

Para el perfil de administración, se han desarrollado tres herramientas visuales complementarias que permiten una supervisión global de la plantilla:

- **Listado de gestión:** Un panel para la vista rápida de solicitudes pendientes (véase figura 12).
- **Calendario mensual:** Una vista para identificar rápidamente qué trabajadores se encuentran ausentes en una fecha concreta (véase figura 12).
- **Cuadrante:** Una matriz mensual que muestra la situación de todos los empleados, marcando festivos y fines de semana, y calculando el total de días consumidos en tiempo real, tal como en la figura 13.

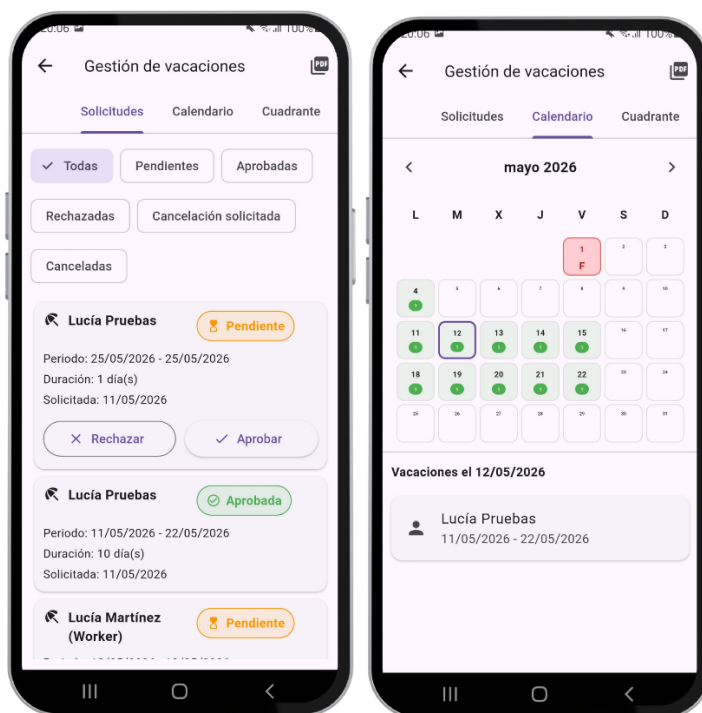


Figura 12. Listado de peticiones y calendario mensual de vacaciones.

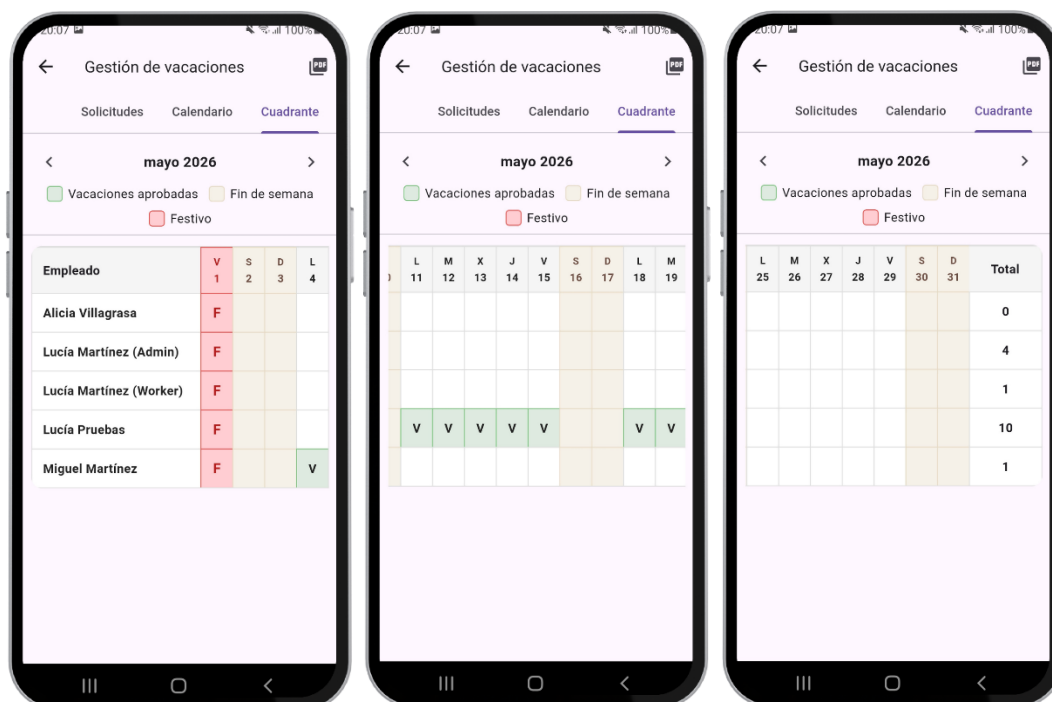


Figura 13. Cuadrante mensual de vacaciones.

Finalmente, también se puede exportar el cuadrante mensual en PDF, como puede verse en la figura 14.

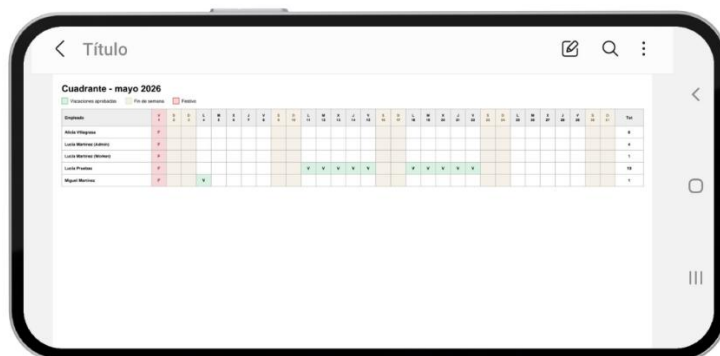


Figura 14. PDF del cuadrante mensual de vacaciones.

6.7 Gestión de incidencias y justificantes

Para dar solución a los posibles errores en el día a día, como olvidos al fichar o ausencias imprevistas, la aplicación incluye un apartado de incidencias y justificantes. Este módulo permite, por un lado, corregir registros horarios de forma supervisada y, por otro, adjuntar documentos que acrediten cualquier ausencia al trabajo.

6.7.1 Incidencias

Las incidencias están diseñadas para corregir olvidos en el fichaje de entradas o salidas. La robustez de este proceso se basa en la aprobación administrativa. Cuando un administrador aprueba una solicitud, el sistema procesa el cambio de estado y la creación del nuevo fichaje como una única operación vinculada. Esto asegura que ambas acciones se realicen correctamente al mismo tiempo. De esta forma, se garantiza que nunca aparezca un fichaje nuevo sin su incidencia correspondiente, manteniendo la base de datos siempre limpia y coherente. Véase figura 15 para conocer la interfaz de este módulo.

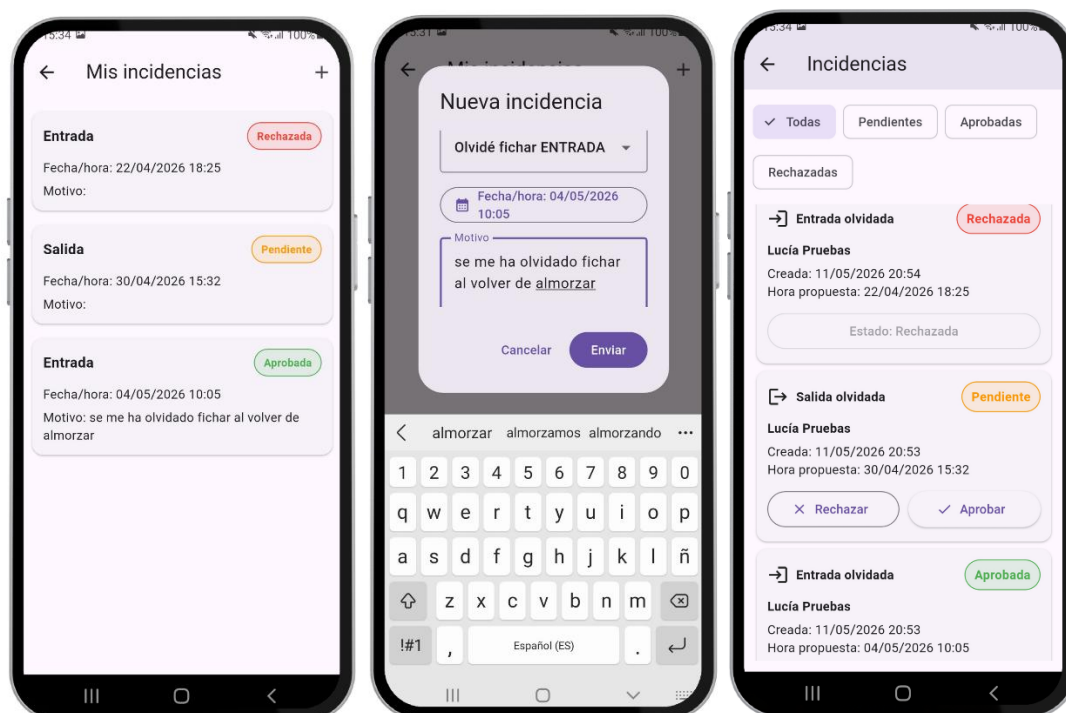


Figura 15. Panel de incidencias y de nueva incidencia del trabajador, y del administrador.

6.7.2 Justificantes

El módulo de justificantes permite al trabajador subir documentos (PDF o imágenes) para respaldar ausencias, y el administrador puede verlos y descargarlos, como se muestra en la figura 16. Se gestionan los archivos y los datos de forma independiente:

- **Firebase Storage:** Repositorio para los archivos físicos.
- **Cloud Firestore:** Almacenamiento exclusivo de los metadatos y la referencia (URL) al archivo en la colección `absence_justifications`.

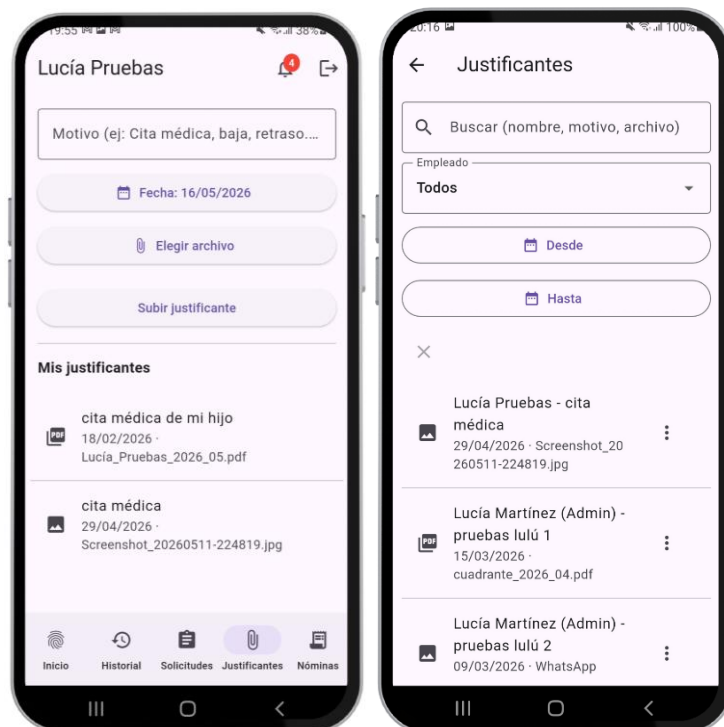


Figura 16. Pantalla de justificantes de trabajador y empleado.

6.7.3 Herramientas de supervisión y privacidad

El acceso a esta información está jerarquizado, mientras que el trabajador solo puede ver sus documentos, el administrador cuenta con un panel de control avanzado. Se han implementado funciones de filtrado múltiple (por rango de fechas, empleado...).

6.8 Gestión de nóminas

Este módulo está centrado en la gestión de nóminas, en este caso, a diferencia del módulo de justificantes, es el administrador quien genera la información y el trabajador quien actúa como receptor. Este proceso garantiza una distribución de documentos oficiales rápida, segura y digitalizada.

Para asegurar un rendimiento óptimo del sistema, se ha mantenido la estrategia de almacenamiento en dos capas. Los archivos PDF se guardan en Firebase Storage [4], mientras que en la colección `payrolls` de Cloud Firestore se registra una entrada con los metadatos (identificador del empleado, periodo (mes y año), ruta y URL de descarga).

La seguridad de la información de este módulo es muy importante puesto que hay información sobre datos económicos. Por ello, el acceso se ha restringido:

- **Perfil de Trabajador:** La aplicación aplica un filtro automático que impide visualizar cualquier documento que no esté vinculado a su identificador.

- **Perfil de Administrador:** Desde esta vista se pueden consultar las nóminas de toda la plantilla.

Ambas interfaces pueden verse en la figura 17.

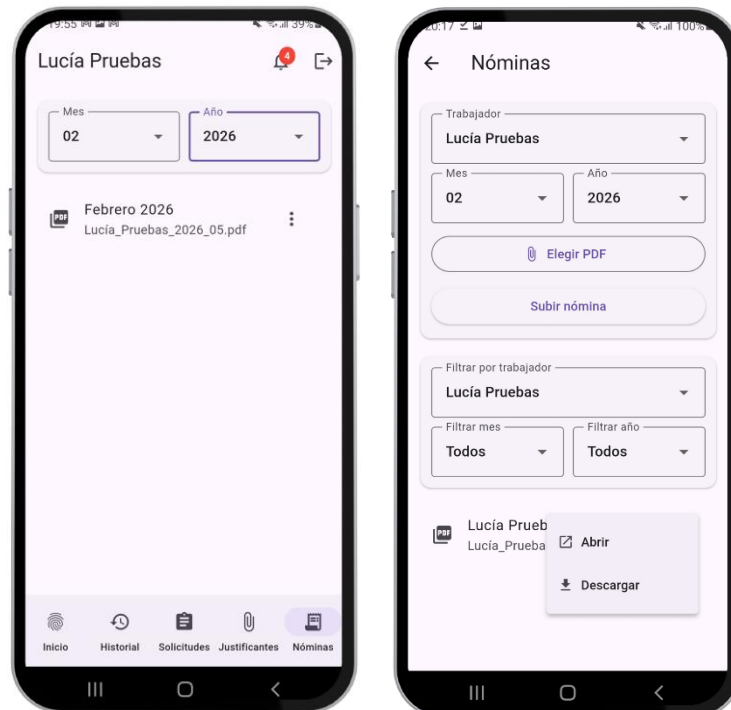


Figura 17. Pantalla de nóminas de trabajador y de administrador.

6.9 Sistema de notificaciones

Para garantizar una buena comunicación entre administradores y trabajadores, se ha desarrollado un módulo de notificaciones internas. Los usuarios reciben actualizaciones al instante sobre cualquier cambio relevante.

6.9.1 Alertas por perfil

La lógica de notificaciones se ha diseñado de forma bidireccional, adaptándose a las necesidades de cada rol:

- **Para el trabajador:** Recibe alertas automáticas cuando el administrador resuelve sus peticiones (vacaciones o incidencias) o cuando se publica su nómina mensual.
- **Para el administrador:** Se generan avisos en el momento en que un empleado registra una nueva solicitud que requiere validación.

6.9.2 Gestión del estado de lectura y persistencia

Estos avisos se gestionan mediante una estructura de datos en Cloud Firestore que permite una sincronización en tiempo real. Esto ofrece varias ventajas técnicas:

- **Control de estado:** El usuario puede distinguir visualmente entre avisos leídos y pendientes, manteniendo un contador actualizado en el icono de la campana.
- **Gestión avanzada de la interfaz:** El usuario puede organizar su bandeja de entrada, marcando como leídas o eliminando notificaciones de forma conjunta o individual, como puede apreciarse en la figura 18.

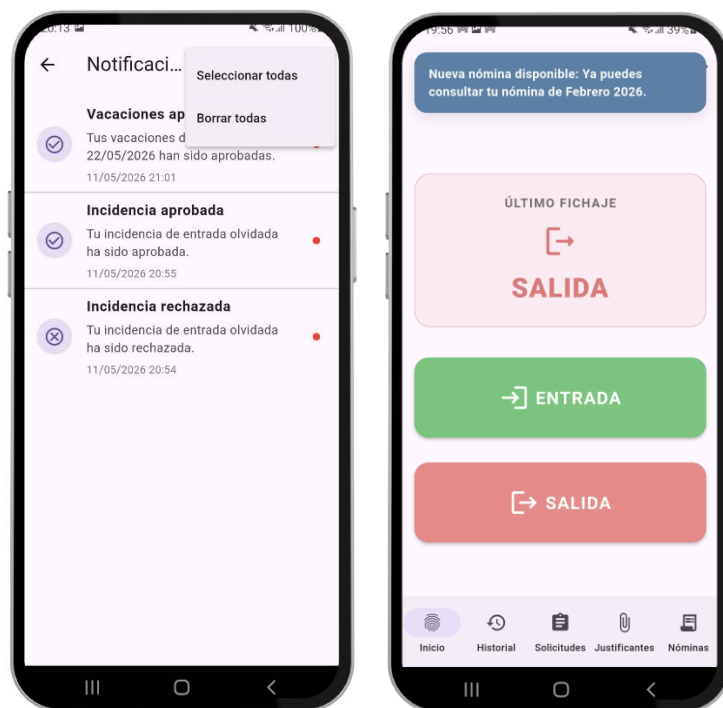


Figura 18. Pantalla de notificaciones y notificación flotante en pantalla.

6.10 Visualización según rol (admin, worker, RLT, inspector)

Para maximizar la eficiencia operativa, la interfaz de la aplicación se reconstruye dinámicamente según el perfil del usuario autenticado. Este enfoque garantiza que cada empleado interactúe exclusivamente con las herramientas relativas a sus funciones.

6.10.1 Configuración automática del entorno

Nada más completar el acceso, la aplicación evalúa el perfil del usuario para personalizar la navegación. Este proceso permite que una única base de código ofrezca experiencias de uso totalmente diferenciadas:

- **Perfil de Administración:** La interfaz muestra el acceso a los módulos de gestión de empleados y asignación de roles, control horario de la plantilla y gestión de solicitudes de vacaciones, incidencias y nóminas. Además, se ha habilitado un modo de "vista de empleado" para que el administrador pueda validar la experiencia de usuario sin cambiar de cuenta. Todas estas funcionalidades pueden verse reflejadas en la figura 19.
- **Perfil de Trabajador:** El menú de navegación se reduce a las funciones de fichaje diario, consulta de historial, solicitudes de vacaciones e incidencias, carga de justificantes y consulta de nóminas, tal y como se observa en la figura 19.
- **Perfiles de Consulta (RLT e Inspección):** Se mantienen las vistas de datos, pero se eliminan botones de edición, aprobación o borrado. La interfaz se transforma en solo lectura, asegurando que la supervisión sea fluida, pero sin riesgo de modificar información. El perfil de Inspección tiene acceso limitado asignado por el administrador. Las pantallas y permisos de RLT se pueden apreciar en la figura 20, y las de Inspección en la Figura 22.

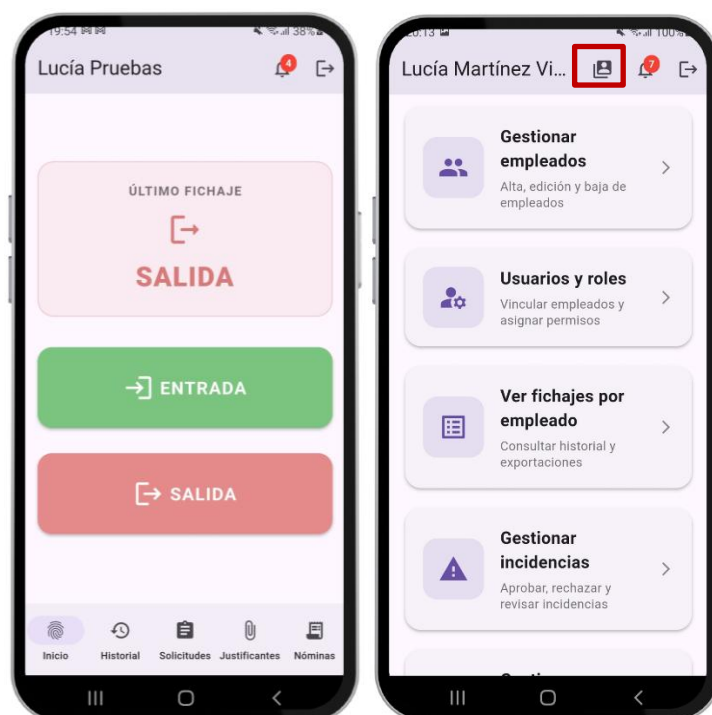


Figura 19. Pantalla principal del trabajador y del administrador.

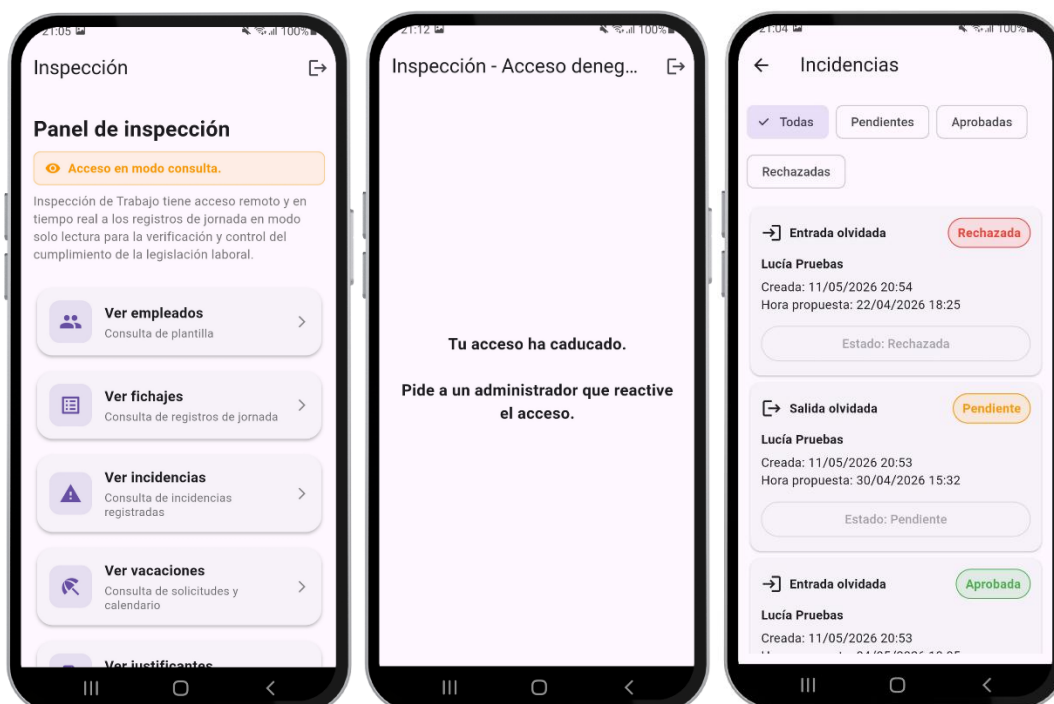


Figura 20. Panel de Inspección con y sin acceso, y vista restringida de un módulo.

La figura 21 representa el acceso habilitado para el rol de inspección, permitiendo la consulta de información laboral en tiempo real.

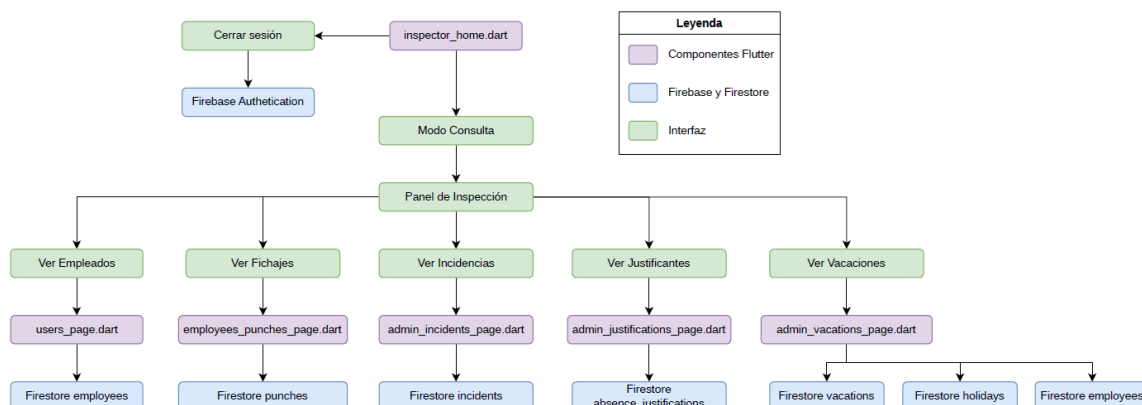


Figura 21. Estructura funcional del panel inspector.

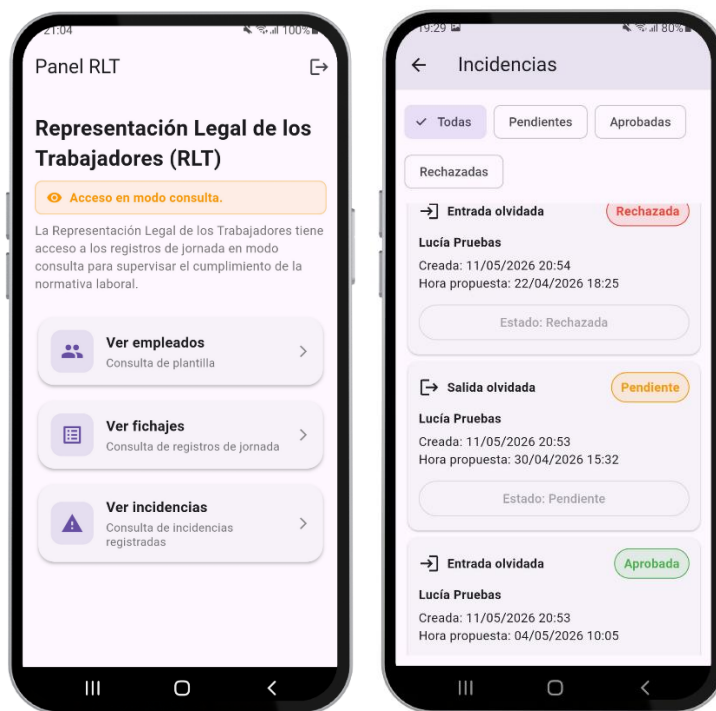


Figura 22. Panel RLT, y vista restringida de un módulo.

La figura 23 muestra la estructura del rol de Representación Legal de los Trabajadores (RLT), configurado en modo solo lectura.

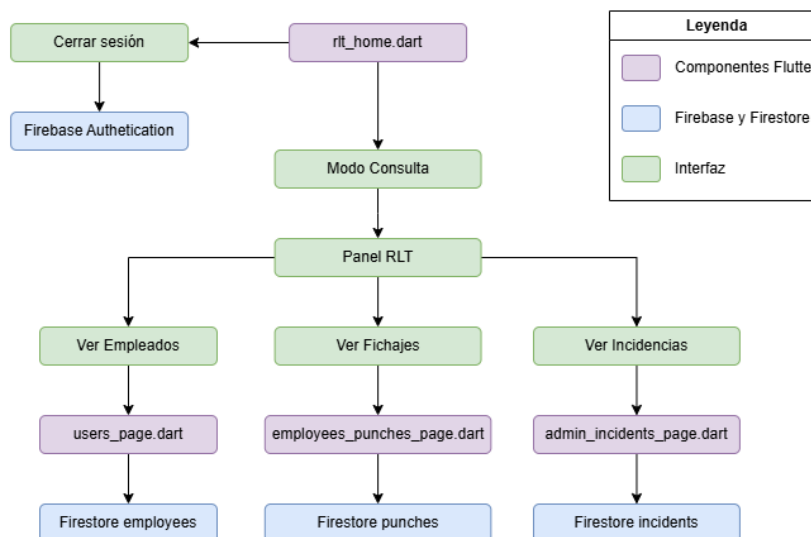


Figura 23. Estructura funcional del panel RLT.

6.11 Funcionamiento en modo offline

Como se ha explicado en la fase de diseño, la aplicación está capacitada para operar con conectividad limitada en el módulo de fichajes. El objetivo es garantizar que la jornada laboral quede registrada sin interrupciones, independientemente de la cobertura del dispositivo.

6.11.1 Gestión del estado de sincronización

Cuando se detecta una falta de conexión, la aplicación usa la persistencia local para almacenar el registro. Para evitar la duda del usuario, el sistema implementa una lógica de *feedback* visual:

- **Identificación visual:** Mientras el fichaje solo existe en la memoria local del dispositivo, al fichar sale un mensaje indicando que se ha guardado el fichaje sin conexión, y se muestra un icono de nube junto al registro, tal y como se ve en la figura 24.
- **Sincronización automática:** En el momento en que se restablece la conexión, las librerías de persistencia de Firebase [4] suben los documentos pendientes. Una vez confirmada la persistencia en la nube, el icono desaparece automáticamente, notificando de forma implícita que ya es accesible para la administración.

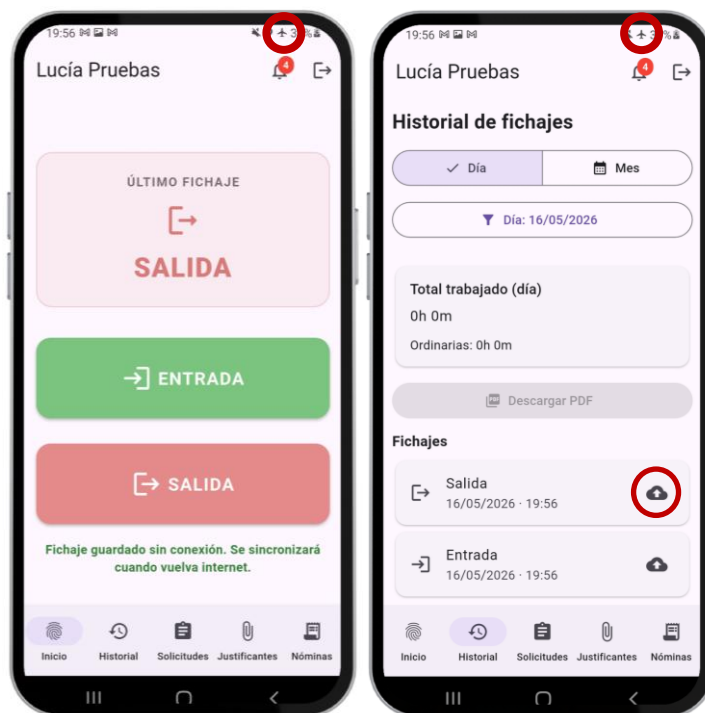


Figura 24. Mensaje “fichaje guardado sin conexión” e icono de la nube en el registro.

6.11.2 Diferencias entre plataformas

Esta capacidad está diseñada para la aplicación móvil. Debido a las limitaciones propias de los navegadores y a la arquitectura de Firebase Hosting [4], la versión web está configurada inicialmente para trabajar con conexión, asegurando así que los datos se guarden envíen directamente al servidor sin depender del almacenamiento del navegador.

Capítulo 7. Tests y resultados

7.1 Tests unitarios

Con el objetivo de garantizar la estabilidad del software y verificar la integridad de la lógica, se ha diseñado una batería de pruebas unitarias automatizadas mediante el paquete nativo flutter_test. Estas pruebas permiten asegurar que las funciones críticas del sistema respondan correctamente ante diferentes escenarios de uso y conjuntos de datos controlados.

Tras su ejecución, los resultados han confirmado que el 100% de los casos de prueba diseñados han sido superados con éxito, validando la robustez de la arquitectura implementada.

La tabla 2 detalla el desglose de las pruebas unitarias integradas junto a sus respectivos objetivos de verificación.

ID	Elemento probado	Descripción	Resultado esperado	Estado
TU-01	UserRoles.admin	Verificación del valor del rol administrador	El rol administrador devuelve correctamente el valor "admin"	Superado
TU-02	UserRoles.worker	Verificación del valor del rol trabajador	El rol trabajador devuelve correctamente el valor "worker"	Superado
TU-03	UserRoles.all	Comprobación de inclusión del rol inspector	La lista de roles contiene correctamente el rol inspector	Superado
TU-04	UserRoles.label()	Conversión de rol administrador a etiqueta legible	El sistema devuelve "Administrador"	Superado
TU-05	UserRoles.label()	Conversión de rol trabajador a etiqueta legible	El sistema devuelve "Trabajador"	Superado
TU-06	UserRoles.label()	Conversión de rol inspector a etiqueta legible	El sistema devuelve "Inspección"	Superado
TU-07	UserRoles.label()	Comprobación de rol desconocido	El sistema devuelve el valor original del rol	Superado
TU-08	WorkTime.dayKey()	Eliminación de hora en fechas	El sistema conserva únicamente año, mes y día	Superado
TU-09	WorkTime.dayLabel()	Formateo de fecha	La fecha se muestra en formato dd/MM/yyyy	Superado
TU-10	WorkTime.minutesByDay()	Cálculo de jornada laboral	El sistema calcula correctamente 480 minutos trabajados	Superado
TU-11	WorkTime.ordinaryMinutes()	Cálculo de horas ordinarias	El sistema limita correctamente las horas ordinarias a 480 minutos	Superado
TU-12	WorkTime.extraMinutes()	Cálculo de horas extra	El sistema calcula correctamente los minutos extra	Superado

TU-13	WorkTime.formatHM()	Conversión de minutos a horas y minutos	El sistema devuelve correctamente el formato "2h 5m"	Superado
TU-14	VacationRequest.copyWith()	Actualización parcial de solicitudes	El sistema modifica únicamente los campos indicados	Superado
TU-15	VacationRequest.toMap()	Conversión del modelo a mapa Firestore	El sistema genera correctamente la estructura de datos	Superado
TU-16	VacationRequest.toMap()	Normalización de fechas	Las fechas se almacenan sin componente horaria	Superado
TU-17	ExportService.buildFileName()	Generación de nombre de archivo CSV	El sistema genera correctamente el nombre del archivo	Superado
TU-18	ExportService.buildFileNameForMonth()	Generación de nombre de archivo mensual PDF	El sistema genera correctamente el nombre mensual del archivo	Superado
TU-19	ExportService.buildCsvBytes()	Exportación de fichajes a CSV	El sistema genera correctamente el contenido CSV	Superado
TU-20	AppNotification.isRead	Comprobación de notificación no leída	El sistema detecta correctamente notificaciones pendientes	Superado
TU-21	AppNotification.isRead	Comprobación de notificación leída	El sistema detecta correctamente notificaciones leídas	Superado
TU-22	flutter_test	Ejecución global de tests automatizados	Todos los tests automatizados se ejecutan correctamente	Superado

Tabla 2. Pruebas unitarias.

7.2 Tests funcionales

Las pruebas funcionales se han realizado con el objetivo de comprobar el correcto funcionamiento de las distintas funcionalidades implementadas en la aplicación desde el punto de vista del usuario final.

Estas pruebas se han llevado a cabo manualmente utilizando los diferentes roles disponibles en el sistema, verificando tanto el comportamiento de cada módulo como la respuesta de la interfaz ante las distintas acciones.

La tabla 3 recoge las principales pruebas funcionales realizadas durante el desarrollo del proyecto.

ID	Funcionalidad	Pasos realizados	Resultado esperado	Resultado obtenido	Estado
PF-01	Inicio de sesión correcto	Abrir la aplicación, introducir email y contraseña válidos, pulsar en iniciar sesión	El usuario accede correctamente a la pantalla principal	El usuario accedió correctamente al panel principal del administrador	Superado

			correspondiente a su rol		
PF-02	Inicio de sesión con contraseña incorrecta	Abrir la aplicación, introducir un email válido, introducir una contraseña incorrecta, pulsar en iniciar sesión	El sistema muestra un mensaje de error y no permite acceder a la aplicación	El sistema mostró el mensaje "Correo o contraseña incorrectos" y permaneció en la pantalla de inicio de sesión.	Superado
PF-03	Inicio de sesión con campos vacíos	Abrir la aplicación, dejar los campos de email y contraseña vacíos, pulsar en iniciar sesión	El sistema muestra mensajes de validación indicando que los campos son obligatorios	El sistema mostró el mensaje "Introduce el correo y la contraseña"	Superado
PF-04	Recuperación de contraseña	Abrir la pantalla de recuperación de contraseña, introducir un email válido, pulsar enviar	El sistema muestra un mensaje indicando que el correo de recuperación ha sido enviado	El sistema mosntró el mensaje "Email enviado correctamente. Revisa tambien la carpeta de Spam o Correo no deseado" y envió el email.	Superado
PF-05	Cierre de sesión	Iniciar sesión en la aplicación, pulsar el botón de cerrar sesión	El sistema cierra la sesión y vuelve a la pantalla de inicio de sesión	El sistema accedió correctamente al panel principal del administrador, y volvió al panel de inicio de sesión.	Superado
PF-06	Redirección de administrador	Iniciar sesión con un usuario administrador	El sistema muestra el panel de administración	El sistema accedió correctamente al panel principal del administrador	Superado
PF-07	Redirección de trabajador	Iniciar sesión con un usuario trabajador	El sistema muestra el panel principal del trabajador	El sistema accedió correctamente al panel principal del trabajador	Superado

PF-08	Redirección de RLT	Iniciar sesión con un usuario con rol RLT	El sistema muestra el panel correspondiente al representante de los trabajadores	El sistema accedió correctamente al panel principal del Representante Legal de los Trabajadores	Superado
PF-09	Redirección de inspector	Iniciar sesión con un usuario inspector con acceso activo	El sistema muestra el panel de inspección	El sistema accedió correctamente al panel principal de Inspección	Superado
PF-10	Registro de entrada	Iniciar sesión como trabajador, pulsar el botón de fichar entrada	El sistema registra correctamente la hora de entrada	El sistema registra correctamente la hora de entrada en historial	Superado
PF-11	Registro de salida	Tras haber registrado una entrada, pulsar el botón de fichar salida	El sistema registra correctamente la hora de salida	El sistema registra correctamente la hora de salida en historial	Superado
PF-12	Visualización del historial de fichajes	Acceder al historial de fichajes desde el panel del trabajador	El sistema muestra correctamente el listado de fichajes realizados	El sistema muestra el historial de fichajes	Superado
PF-13	Cálculo de horas trabajadas	Registrar una entrada y una salida válidas	El sistema calcula correctamente el tiempo trabajado	El sistema calcula correctamente el total de horas trabajadas	Superado
PF-14	Fichaje sin conexión	Realizar un fichaje sin conexión a internet	El sistema almacena el fichaje y lo sincroniza posteriormente	El sistema almacenó el fichaje y se sincronizó al recuperar la conexión	Superado
PF-15	Creación de incidencia	Acceder al apartado de incidencias, rellenar el formulario, enviar incidencia	El sistema registra correctamente la incidencia	El sistema guarda correctamente el registro de nueva incidencia	Superado

PF-16	Visualización de incidencias	Acceder al listado de incidencias registradas	El sistema muestra correctamente las incidencias almacenadas	El sistema muestra el listado de incidencias y sus estados	Superado
PF-17	Resolución de incidencia por administrador	Acceder como administrador, aceptar o rechazar una incidencia	El sistema actualiza correctamente el estado de la incidencia	El sistema actualiza el estado de las incidencias	Superado
PF-18	Notificación de resolución de incidencia	Resolver una incidencia desde administración	El trabajador recibe una notificación con el resultado de la incidencia	El trabajador recibe una notificación con el cambio de estado de la incidencia	Superado
PF-19	Solicitud de vacaciones	Acceder al apartado de vacaciones, seleccionar fechas, enviar solicitud	El sistema registra correctamente la solicitud de vacaciones	El sistema guarda correctamente el registro de nueva petición de vacaciones	Superado
PF-20	Visualización de solicitudes de vacaciones	Acceder al historial de solicitudes de vacaciones	El sistema muestra correctamente las solicitudes registradas	El sistema muestra el listado de peticiones de vacaciones y sus estados.	Superado
PF-21	Aprobación de vacaciones	Acceder como administrador, aprobar una solicitud de vacaciones	El sistema actualiza correctamente el estado de la solicitud	El sistema actualiza el estado de las vacaciones a aprobadas	Superado
PF-22	Rechazo de vacaciones	Acceder como administrador, rechazar una solicitud de vacaciones	El sistema actualiza correctamente el estado de la solicitud	El sistema actualiza el estado de las vacaciones a rechazadas	Superado
PF-23	Subida de justificante	Acceder al apartado de justificantes, seleccionar un archivo, subir justificante	El sistema almacena correctamente el justificante	El sistema guarda el justificante	Superado
PF-24	Visualización de justificantes	Acceder al listado de justificantes registrados	El sistema muestra correctamente	El sistema muestra los justificantes almacenados	Superado

			los justificantes almacenados		
PF-25	Acceso a inspección con permiso caducado	Iniciar sesión con un inspector con acceso caducado, acceder al panel de inspección	El sistema muestra un mensaje indicando que el acceso ha caducado	El sistema no permite acceder al panel y muestra un mensaje de "Tu acceso ha caducado. Pide a un administrador que reactive el acceso"	Superado
PF-26	Visualización de empleados en inspección	Acceder al panel de inspección con permisos válidos	El sistema muestra correctamente la información disponible para inspección	El sistema muestra la información de los empleados, pero sin poder modificarla	Superado
PF-27	Permisos del rol RLT	Iniciar sesión con un usuario RLT, acceder a funcionalidades restringidas	El sistema únicamente permite acceder a las funciones autorizadas para el rol RLT	El sistema muestra la información permitida, pero no deja modificarla.	Superado

Tabla 3. Pruebas funcionales.

7.3 Tests de integración

Las pruebas funcionales realizadas durante el desarrollo también han permitido validar la correcta integración entre la aplicación desarrollada en Flutter [5] y los servicios de Firebase [4] utilizados como *backend*.

Durante estas comprobaciones se ha verificado el correcto funcionamiento conjunto de Firebase Authentication, Cloud Firestore y Firebase Storage [4] en operaciones de autenticación, almacenamiento, sincronización y consulta de información.

Además, se ha comprobado la correcta actualización de datos entre los distintos módulos de la aplicación, incluyendo fichajes, incidencias, vacaciones, justificantes y notificaciones internas.

7.4 Tests de usabilidad y compatibilidad

Las pruebas de usabilidad y compatibilidad se han realizado con el objetivo de verificar la experiencia de uso general de la aplicación y su correcto funcionamiento en distintos entornos de ejecución.

La tabla 4 recoge las principales pruebas de usabilidad y compatibilidad realizadas.

ID	Tipo	Funcionalidad	Pasos realizados	Resultado esperado	Resultado obtenido	Estado
PUC-01	Usabilidad	Navegación entre pantallas	Navegar entre las distintas pantallas	La navegación se realiza de	La navegación funciona	Superado

			de la aplicación	forma fluida e intuitiva	correctamente sin errores	
PUC-02	Compatibilidad	Visualización en dispositivo móvil	Utilizar la aplicación desde un dispositivo móvil	La interfaz se adapta correctamente al tamaño de pantalla	La interfaz de adapta al tamaño del móvil	Superado
PUC-03	Compatibilidad	Visualización en Flutter Web	Utilizar la aplicación desde navegador web	La aplicación funciona correctamente en entorno web	La interfaz de adapta al tamaño del ordenador	Superado

Tabla 4. Pruebas de usabilidad y compatibilidad.

7.5 Resultados obtenidos

La solución desarrollada constituye una herramienta operativa para la supervisión horaria y la gestión administrativa del personal. El sistema unifica en un único entorno los módulos de registro de jornada, tramitación de incidencias, solicitudes de vacaciones, validación de justificantes y un control de accesos basado en roles.

La solución implementada centraliza los flujos operativos de la empresa mediante una interfaz común que responde correctamente tanto en dispositivos móviles como en navegadores web.

Finalmente, la arquitectura de software adoptada demuestra ser estable frente a excepciones y escalable ante futuras ampliaciones, ofreciendo una experiencia de usuario intuitiva, fluida y adaptada a las necesidades específicas de cada perfil de la organización.

Capítulo 8. Pliego de condiciones

8.1 Condiciones de funcionamiento

La aplicación se ha desarrollado bajo un enfoque multiplataforma, lo que permite que el código base sea compatible tanto con Android como con iOS. Sin embargo, las pruebas de ejecución se han realizado exclusivamente en dispositivos Android. Esto se debe a que el ecosistema de Apple requiere obligatoriamente de hardware con macOS y la herramienta Xcode para compilar y desplegar la aplicación, recursos que no han estado disponibles durante el desarrollo de este proyecto.

8.2 Estimación de costes

Aunque este proyecto ha sido realizado con fines académicos, se presenta a continuación un análisis de lo que supondría su puesta en marcha en un entorno profesional.

Se estima una inversión total de entre 300 - 400 horas de trabajo, distribuidas entre el análisis previo, el diseño de la arquitectura, el desarrollo de las funcionalidades y la fase de pruebas. Aplicando un coste medio de 20 €/hora, el valor del desarrollo ascendería a entre 6.000 y 8.000 €.

Para este proyecto se ha seleccionado el Plan Blaze de Firebase. A diferencia del plan gratuito básico, el Plan Blaze permite el uso de Firebase Storage para la gestión de documentos (nóminas y justificantes) y ofrece mayores límites operativos [4].

Este plan usa el modelo *pay-as-you-go*, cuya gran ventaja es que el coste es proporcional al uso real. Dado que el volumen de tráfico y almacenamiento durante la fase de desarrollo ha sido bajo, los costes facturados han sido insignificantes. No obstante, este plan asegura que la aplicación estaría lista para escalar si el número de trabajadores aumentara.

Capítulo 9. Conclusiones

9.1 Conclusiones

El desarrollo de este Trabajo Fin de Grado me ha permitido diseñar e implementar una solución móvil multiplataforma que responde a la necesidad real de modernizar el control horario en las empresas. A lo largo del proceso, he conseguido integrar módulos complejos como la gestión de vacaciones, el registro de incidencias, los justificantes, la consulta de nóminas y un sistema de notificaciones en tiempo real, todo ello bajo una estructura de permisos por roles que garantiza la seguridad de la información.

La elección de Flutter y Firebase [2] ha sido clave, ya que el primero me ha permitido desarrollar una base de código única capaz de ejecutarse tanto en Android como en iOS, optimizando los tiempos de desarrollo, y el segundo, ha simplificado la infraestructura, puesto que no ha sido necesario configurar ni mantener un servidor propio.

Además, la aplicación ha sido diseñada teniendo en cuenta la evolución de la normativa laboral española en materia de control horario digital [6], [7]. Debido a que la aprobación definitiva de estas leyes está pendiente, y que aún no están disponibles los requisitos detallados, la implementación actual tiene un carácter experimental.

A nivel personal, este proyecto ha supuesto un reto importante: no solo he consolidado mis conocimientos en el desarrollo móvil y bases de datos NoSQL, aprendidos durante el grado, y más precisamente durante mi Erasmus, sino que también he aprendido a gestionar la arquitectura de servicios Cloud y a resolver los problemas lógicos que surgen en un sistema de esta complejidad.

9.2 Limitaciones

A pesar de haber cumplido los objetivos principales, el proyecto presenta ciertas limitaciones derivadas del contexto académico y de los recursos técnicos disponibles:

- **Restricciones en iOS:** Aunque la aplicación es técnicamente compatible con el ecosistema de Apple gracias a Flutter [5], no se ha podido comprobar su funcionamiento en dispositivos iPhone. Esta limitación surge de la necesidad de disponer de hardware específico (macOS) y la herramienta Xcode para la compilación, recursos a los que no se ha tenido acceso durante el desarrollo.
- **Arquitectura de base de datos única (Single-tenant):** Una limitación clave para un posible despliegue comercial en App Store o Google Play es que la aplicación está vinculada directamente a una instancia específica de Firebase [4], lo que haría que diferentes empresas compartieran la misma base de datos. Actualmente, para que una nueva empresa utilizara el sistema, sería necesario realizar una intervención manual para enlazarla a una base de datos propia. Sin embargo, este diseño se eligió deliberadamente para una prueba de concepto, ya que el objetivo actual es demostrar que el sistema funciona. Al no ser un fallo de diseño, la transición a un modelo *multi-tenant* se plantea como una mejora técnica a implementar en el futuro.
- **Escalabilidad en el entorno real:** Si bien el uso del Plan Blaze de Firebase [4] elimina la necesidad de gestionar un servidor propio y permite un crecimiento flexible, el sistema no ha sido sometido a pruebas de carga con miles de usuarios. En un entorno de producción masivo, sería necesario realizar optimizaciones adicionales en las consultas a la base de datos para controlar los costes operativos del modelo "pago por uso".
- **Validación legal:** Aunque la aplicación cumple con la lógica de registro de jornada y almacenamiento de documentos, no ha sido sometida a una auditoría legal específica para certificar que cumple con todos los requisitos de seguridad y protección de datos exigidos.

9.3 Trabajo futuro

El sistema actual sienta las bases de una herramienta funcional de gestión laboral, pero su arquitectura permite proyectar diversas líneas de mejora para convertirlo en un producto comercial de alto rendimiento:

- **Persistencia Offline integral:** Aunque actualmente el registro de fichajes soporta el modo offline, una evolución natural sería extender esta capacidad a toda la aplicación. Esto permitiría a los usuarios consultar sus nóminas ya descargadas, revisar sus vacaciones o generar incidencias sin necesidad de cobertura, sincronizando todos los cambios de forma masiva al recuperar la conexión.
- **Implementación avanzada de notificaciones (FCM):** Se plantea la integración de Firebase Cloud Messaging (FCM) para gestionar notificaciones *push* nativas. Esto permitiría enviar notificaciones en tiempo real incluso con la aplicación cerrada, avisando instantáneamente al trabajador sobre la aprobación de sus vacaciones o al administrador sobre una nueva incidencia urgente [4].
- **Evolución hacia una arquitectura *Multi-tenant*:** Para transformar el proyecto en una solución comercial que se pueda distribuir en tiendas oficiales, el principal paso técnico sería rediseñar la base de datos para soportar múltiples organizaciones de forma aislada. Esto permitiría que cualquier empresa pudiera registrarse y utilizar su propio entorno privado sin necesidad de configuraciones manuales en el *backend*.
- **Validación y despliegue en iOS:** Una línea prioritaria es la obtención de un entorno de compilación en macOS para generar y validar la versión de la app en iOS, garantizando que la experiencia de usuario sea idéntica en ambos ecosistemas.
- **Seguridad Biométrica:** Se propone el uso de sensores de huella dactilar o reconocimiento facial para el inicio de sesión añadiendo una capa de validez legal y seguridad extra a los trámites administrativos.

Capítulo 10. Bibliografía

- [1] Ministerio de Trabajo y Economía Social, “Guía sobre el registro de jornada,” <https://www.mites.gob.es/ficheros/ministerio/GuiaRegistroJornada.pdf>. [Online].
- [2] Google, “Flutter documentation,” <https://firebase.google.com/docs/flutter?hl=es-419>. [Online].
- [3] Google, “Dart language tour and learning resources.” <https://dart.dev/learn>. [Online].
- [4] Digital55, “Qué es Firebase: funcionalidades, ventajas y conclusiones.” <https://digital55.com/blog/que-es-firebase-funcionalidades-ventajas-conclusiones/>. [Online].
- [5] Flutter, “Flutter documentation,” <https://docs.flutter.dev/>. [Online].
- [6] Control Laboral, “Nueva ley de control horario de trabajadores.” <https://controllaboral.es/nueva-ley-control-horario-trabajadores/>. [Online].
- [7] Kronjop, “Ley de control horario.” <https://kronjop.com/es/newsroom/control-horario/ley/>. [Online].

Capítulo 11. Anexos

11.1 Reglas de seguridad de Cloud Firestore

La seguridad y privacidad de la base de datos se gestiona mediante reglas de acceso en Cloud Firestore. Estas reglas controlan quién puede entrar a cada colección según el usuario autenticado y su rol en el sistema, estableciendo así un control de accesos basado en permisos.

A continuación, se muestran las figuras 25 a 33 con estas reglas, las cuales regulan el acceso a secciones como usuarios, empleados, fichajes, incidencias, vacaciones, justificantes, nóminas y notificaciones.

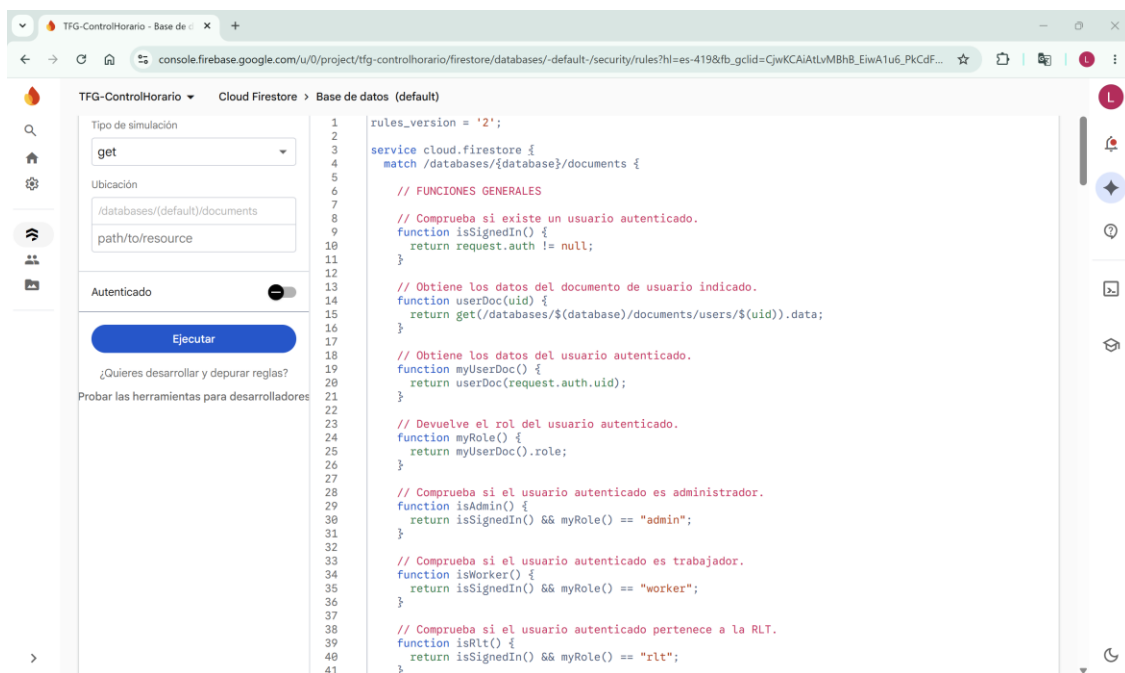
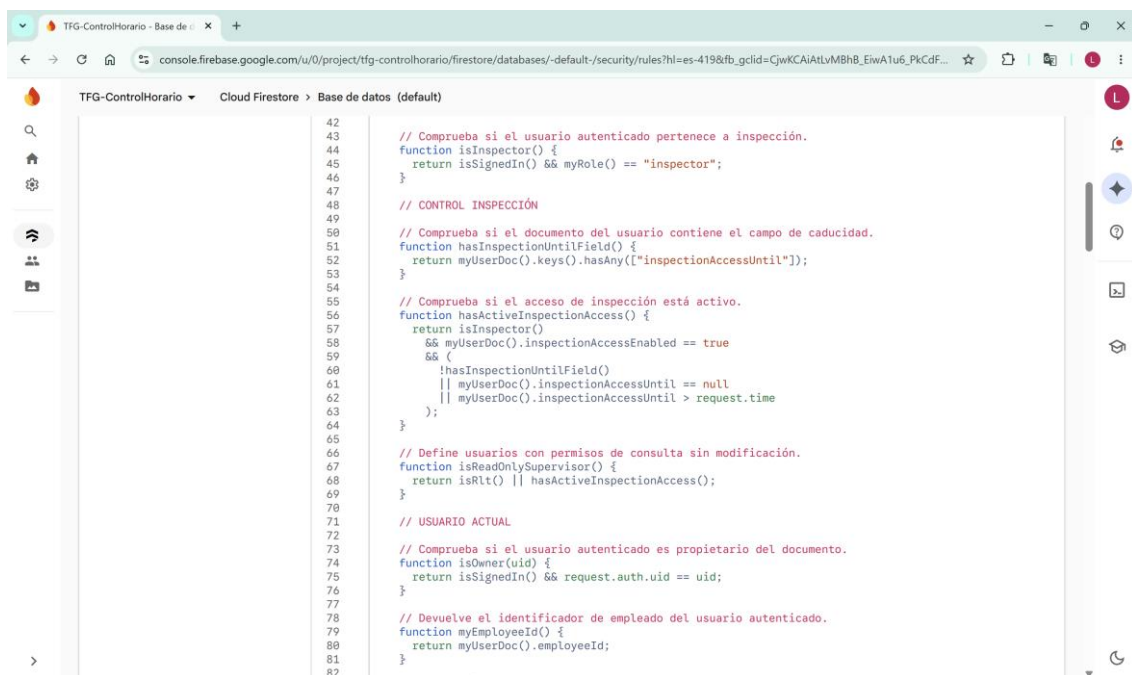


Figura 25. Reglas de acceso sobre las funciones generales.

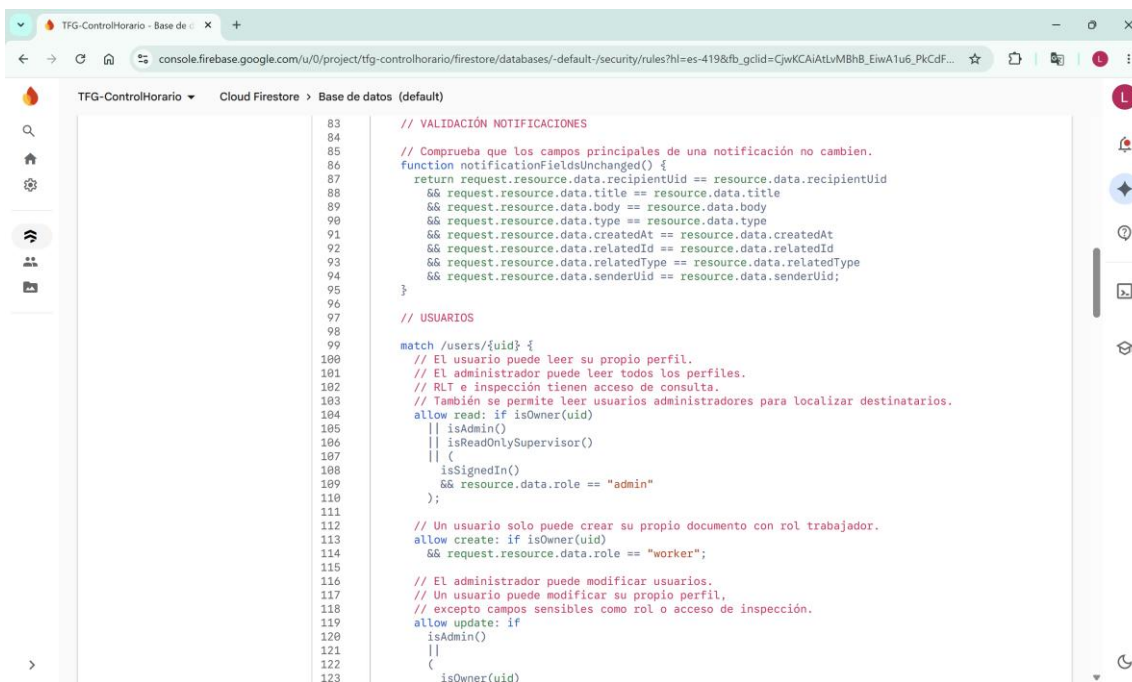


```

42
43 // Comprueba si el usuario autenticado pertenece a inspección.
44 function isInspector() {
45   return isSignedIn() && myRole() == "inspector";
46 }
47
48 // CONTROL INSPECCIÓN
49
50 // Comprueba si el documento del usuario contiene el campo de caducidad.
51 function hasInspectionUntilField() {
52   return myUserDoc().keys().hasAny(["inspectionAccessUntil"]);
53 }
54
55 // Comprueba si el acceso de inspección está activo.
56 function hasActiveInspectionAccess() {
57   return isInspector()
58     && myUserDoc().inspectionAccessEnabled == true
59     && (
60       !hasInspectionUntilField()
61       || myUserDoc().inspectionAccessUntil == null
62       || myUserDoc().inspectionAccessUntil > request.time
63     );
64 }
65
66 // Define usuarios con permisos de consulta sin modificación.
67 function isReadOnlySupervisor() {
68   return isRt() || hasActiveInspectionAccess();
69 }
70
71 // USUARIO ACTUAL
72
73 // Comprueba si el usuario autenticado es propietario del documento.
74 function isOwner(uid) {
75   return isSignedIn() && request.auth.uid == uid;
76 }
77
78 // Devuelve el identificador de empleado del usuario autenticado.
79 function myEmployeeId() {
80   return myUserDoc().employeeId;
81 }
82

```

Figura 26. Reglas de acceso sobre el control de inspección y el usuario actual.

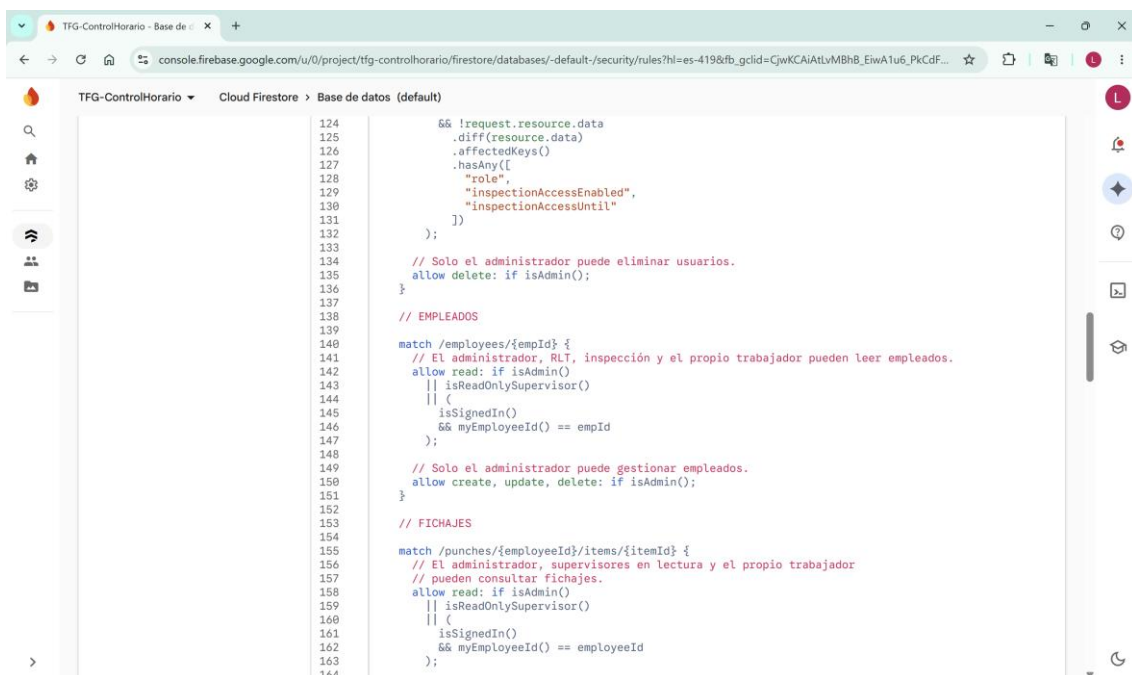


```

83 // VALIDACIÓN NOTIFICACIONES
84
85 // Comprueba que los campos principales de una notificación no cambien.
86 function notificationFieldsUnchanged() {
87   return request.resource.data.recipientUid == resource.data.recipientUid
88     && request.resource.data.title == resource.data.title
89     && request.resource.data.body == resource.data.body
90     && request.resource.data.type == resource.data.type
91     && request.resource.data.createdAt == resource.data.createdAt
92     && request.resource.data.relatedId == resource.data.relatedId
93     && request.resource.data.relatedType == resource.data.relatedType
94     && request.resource.data.senderUid == resource.data.senderUid;
95 }
96
97 // USUARIOS
98
99 match /users/{uid} {
100   // El usuario puede leer su propio perfil.
101   // El administrador puede leer todos los perfiles.
102   // RLT e inspección tienen acceso de consulta.
103   // También se permite leer usuarios administradores para localizar destinatarios.
104   allow read: if isOwner(uid)
105     || isAdmin()
106     || isReadOnlySupervisor()
107     || (
108       isSignedIn()
109       && resource.data.role == "admin"
110     );
111
112   // Un usuario solo puede crear su propio documento con rol trabajador.
113   allow create: if isOwner(uid)
114     && request.resource.data.role == "worker";
115
116   // El administrador puede modificar usuarios.
117   // Un usuario puede modificar su propio perfil,
118   // excepto campos sensibles como rol o acceso de inspección.
119   allow update: if
120     isAdmin()
121     ||
122     (
123       isOwner(uid)

```

Figura 27. Reglas de acceso sobre la validación de notificaciones y los usuarios.

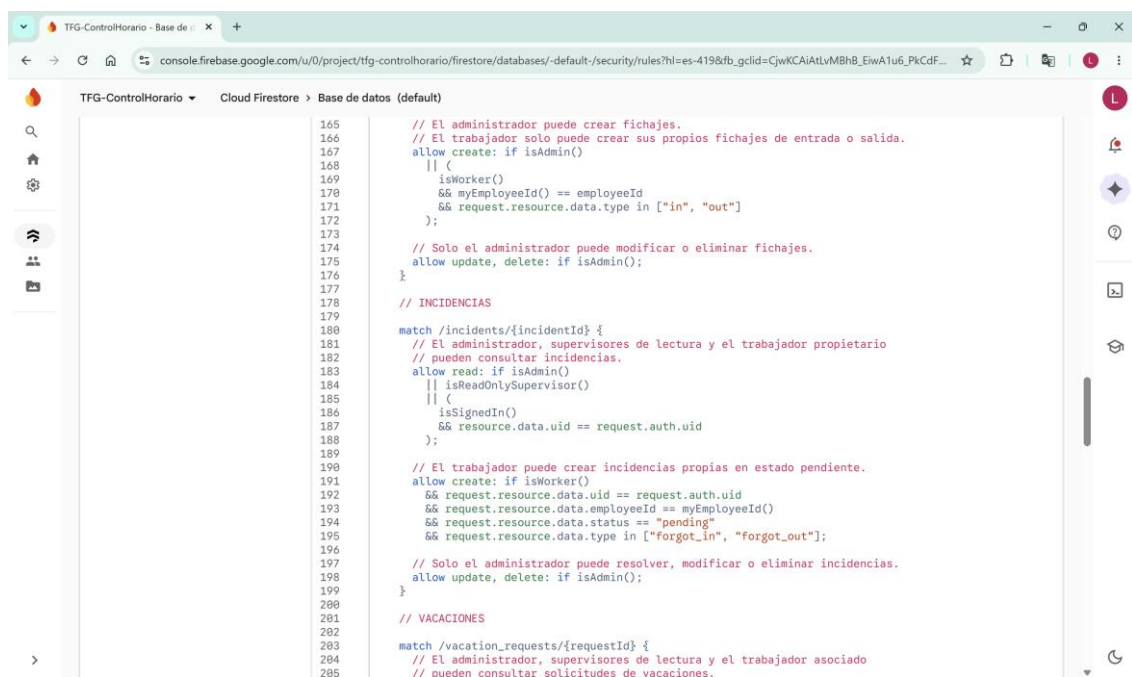


```

124     && !request.resource.data
125       .diff(resource.data)
126       .affectedKeys()
127       .hasAny([
128         "role",
129         "inspectionAccessEnabled",
130         "inspectionAccessUntil"
131       ])
132   );
133
134   // Solo el administrador puede eliminar usuarios.
135   allow delete: if isAdmin();
136 }
137
138 // EMPLEADOS
139
140 match /employees/{empId} {
141   // El administrador, RLT, inspección y el propio trabajador pueden leer empleados.
142   allow read: if isAdmin()
143     || isReadOnlySupervisor()
144     || (
145       isSignedIn()
146       && myEmployeeId() == empId
147     );
148
149   // Solo el administrador puede gestionar empleados.
150   allow create, update, delete: if isAdmin();
151 }
152
153 // FICHAJES
154
155 match /punches/{employeeId}/items/{itemId} {
156   // El administrador, supervisores en lectura y el propio trabajador
157   // pueden consultar fichajes.
158   allow read: if isAdmin()
159     || isReadOnlySupervisor()
160     || (
161       isSignedIn()
162       && myEmployeeId() == employeeId
163     );
164 }

```

Figura 28. Reglas de acceso sobre los empleados y los fichajes.

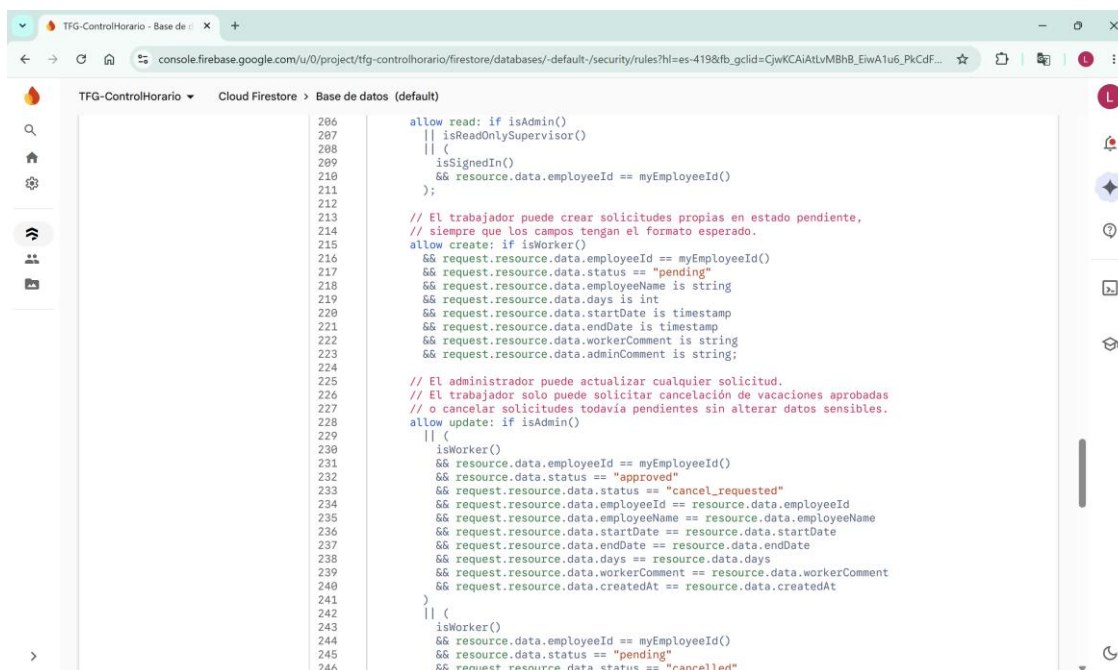


```

165   // El administrador puede crear fichajes.
166   // El trabajador solo puede crear sus propios fichajes de entrada o salida.
167   allow create: if isAdmin()
168     || (
169       isWorker()
170       && myEmployeeId() == employeeId
171       && request.resource.data.type in ["in", "out"]
172     );
173
174   // Solo el administrador puede modificar o eliminar fichajes.
175   allow update, delete: if isAdmin();
176 }
177
178 // INCIDENCIAS
179
180 match /incidents/{incidentId} {
181   // El administrador, supervisores de lectura y el trabajador propietario
182   // pueden consultar incidencias.
183   allow read: if isAdmin()
184     || isReadOnlySupervisor()
185     || (
186       isSignedIn()
187       && resource.data.uid == request.auth.uid
188     );
189
190   // El trabajador puede crear incidencias propias en estado pendiente.
191   allow create: if isWorker()
192     && request.resource.data.uid == request.auth.uid
193     && request.resource.data.employeeId == myEmployeeId()
194     && request.resource.data.status == "pending"
195     && request.resource.data.type in ["forgot_in", "forgot_out"];
196
197   // Solo el administrador puede resolver, modificar o eliminar incidencias.
198   allow update, delete: if isAdmin();
199 }
200
201 // VACACIONES
202
203 match /vacation_requests/{requestId} {
204   // El administrador, supervisores de lectura y el trabajador asociado
205   // pueden consultar solicitudes de vacaciones.

```

Figura 29. Reglas de acceso sobre las incidencias y vacaciones.

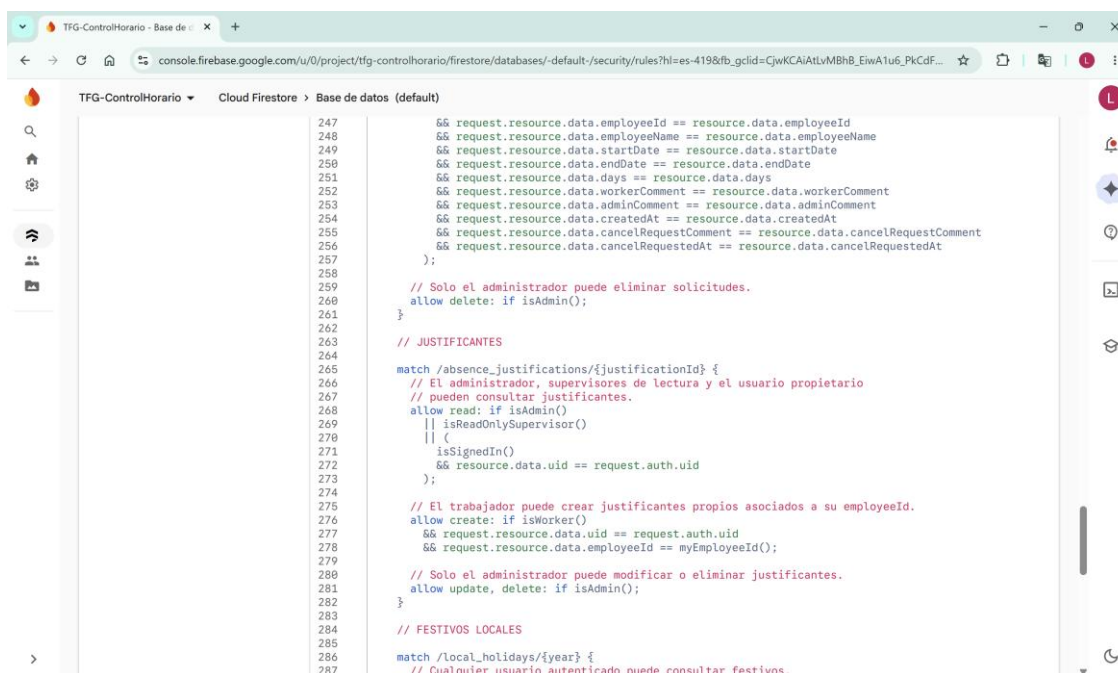


```

206 allow read: if isAdmin()
207 || isReadOnlySupervisor()
208 || (
209   isSignedIn()
210   && resource.data.employeeId == myEmployeeId()
211 );
212
213 // El trabajador puede crear solicitudes propias en estado pendiente,
214 // siempre que los campos tengan el formato esperado.
215 allow create: if isWorker()
216 && request.resource.data.employeeId == myEmployeeId()
217 && request.resource.data.status == "pending"
218 && request.resource.data.employeeName is string
219 && request.resource.data.days is int
220 && request.resource.data.startDate is timestamp
221 && request.resource.data.endDate is timestamp
222 && request.resource.data.workerComment is string
223 && request.resource.data.adminComment is string;
224
225 // El administrador puede actualizar cualquier solicitud.
226 // El trabajador solo puede solicitar cancelación de vacaciones aprobadas
227 // o cancelar solicitudes todavía pendientes sin alterar datos sensibles.
228 allow update: if isAdmin()
229 || (
230   isWorker()
231   && resource.data.status == "approved"
232   && request.resource.data.status == "cancel_requested"
233   && request.resource.data.employeeId == resource.data.employeeId
234   && request.resource.data.employeeName == resource.data.employeeName
235   && request.resource.data.startDate == resource.data.startDate
236   && request.resource.data.endDate == resource.data.endDate
237   && request.resource.data.days == resource.data.days
238   && request.resource.data.workerComment == resource.data.workerComment
239   && request.resource.data.createdAt == resource.data.createdAt
240 )
241 || (
242   isWorker()
243   && resource.data.employeeId == myEmployeeId()
244   && resource.data.status == "pending"
245   && request.resource.data.status == "cancelled"
246 );

```

Figura 30. Reglas de acceso sobre las vacaciones.



```

247 && request.resource.data.employeeId == resource.data.employeeId
248 && request.resource.data.employeeName == resource.data.employeeName
249 && request.resource.data.startDate == resource.data.startDate
250 && request.resource.data.endDate == resource.data.endDate
251 && request.resource.data.days == resource.data.days
252 && request.resource.data.workerComment == resource.data.workerComment
253 && request.resource.data.adminComment == resource.data.adminComment
254 && request.resource.data.createdAt == resource.data.createdAt
255 && request.resource.data.cancelRequestComment == resource.data.cancelRequestComment
256 && request.resource.data.cancelRequestedAt == resource.data.cancelRequestedAt
257 );
258
259 // Solo el administrador puede eliminar solicitudes.
260 allow delete: if isAdmin();
261 }
262
263 // JUSTIFICANTES
264
265 match /absence_justifications/{justificationId} {
266 // El administrador, supervisores de lectura y el usuario propietario
267 // pueden consultar justificantes.
268 allow read: if isAdmin()
269 || isReadOnlySupervisor()
270 || (
271   isSignedIn()
272   && resource.data.uid == request.auth.uid
273 );
274
275 // El trabajador puede crear justificantes propios asociados a su employeeId.
276 allow create: if isWorker()
277 && request.resource.data.uid == request.auth.uid
278 && request.resource.data.employeeId == myEmployeeId();
279
280 // Solo el administrador puede modificar o eliminar justificantes.
281 allow update, delete: if isAdmin();
282 }
283
284 // FESTIVOS LOCALES
285
286 match /local_holidays/{year} {
287 // Cualquier usuario autenticado puede consultar festivos.

```

Figura 31. Reglas de acceso sobre los justificantes y festivos locales.

```

288 allow read: if isSignedIn();
289
290 // Solo el administrador puede gestionar festivos.
291 allow create, update, delete: if isAdmin();
292 }
293
294 // NÓMINAS
295
296 match /payrolls/{payrollId} {
297   // El administrador puede consultar todas las nóminas.
298   // El trabajador solo puede consultar las suyas.
299   allow read: if isAdmin()
300     || (
301       isSignedIn()
302       && resource.data.employeeId == myEmployeeId()
303     );
304
305   // Solo el administrador puede crear, modificar o eliminar nóminas.
306   allow create, update, delete: if isAdmin();
307 }
308
309 // NOTIFICACIONES
310
311 match /notifications/{notificationId} {
312   // El administrador puede leer notificaciones.
313   // Cada usuario puede leer únicamente las que van dirigidas a él.
314   allow read: if isAdmin()
315     || (
316       isSignedIn()
317       && resource.data.recipientUid == request.auth.uid
318     );
319
320   // Cualquier usuario autenticado puede crear una notificación.
321   allow create: if isSignedIn();
322
323   // El destinatario solo puede actualizar campos permitidos,
324   // como marcar la notificación como leída.
325   // El administrador puede actualizar cualquier notificación.
326   allow update: if
327     (
328       isSignedIn()

```

Figura 32. Reglas de acceso sobre las nóminas y notificaciones.

```

303     );
304
305   // Solo el administrador puede crear, modificar o eliminar nóminas.
306   allow create, update, delete: if isAdmin();
307 }
308
309 // NOTIFICACIONES
310
311 match /notifications/{notificationId} {
312   // El administrador puede leer notificaciones.
313   // Cada usuario puede leer únicamente las que van dirigidas a él.
314   allow read: if isAdmin()
315     || (
316       isSignedIn()
317       && resource.data.recipientUid == request.auth.uid
318     );
319
320   // Cualquier usuario autenticado puede crear una notificación.
321   allow create: if isSignedIn();
322
323   // El destinatario solo puede actualizar campos permitidos,
324   // como marcar la notificación como leída.
325   // El administrador puede actualizar cualquier notificación.
326   allow update: if
327     (
328       isSignedIn()
329       && resource.data.recipientUid == request.auth.uid
330       && request.resource.data.recipientUid == resource.data.recipientUid
331       && notificationFieldsUnchanged()
332     )
333     || isAdmin();
334
335   // El administrador o el destinatario pueden eliminar notificaciones.
336   allow delete: if isAdmin()
337     || (
338       isSignedIn()
339       && resource.data.recipientUid == request.auth.uid
340     );
341 }
342
343 }

```

Figura 33. Reglas de acceso sobre las notificaciones.

Estas reglas están diseñadas para cumplir con los siguientes objetivos:

- **Autenticación obligatoria:** Comprobar que el usuario ha iniciado sesión antes de permitir cualquier interacción con la base de datos.
- **Control por roles:** Segmentar los permisos del sistema según el perfil del usuario (administrador, trabajador, RLT o inspector).
- **Privacidad individual:** Restringir el acceso de los empleados para que solo puedan consultar y gestionar su propia información.

- **Restricción de escritura:** Limitar el alcance de ciertos perfiles a permisos exclusivamente de lectura.
- **Protección de datos sensibles:** Blindar documentos críticos como nóminas, notificaciones o registros del sistema frente a modificaciones no autorizadas.

11.2 Reglas de seguridad de Firebase Storage

Firebase Storage gestiona el almacenamiento de los archivos de la aplicación (nóminas y justificantes). Para proteger esta información, se han configurado reglas de acceso específicas que comprueban la identidad y el rol del usuario antes de autorizar cualquier lectura o escritura. Las figuras 34 y 35 muestran estas reglas de acceso.

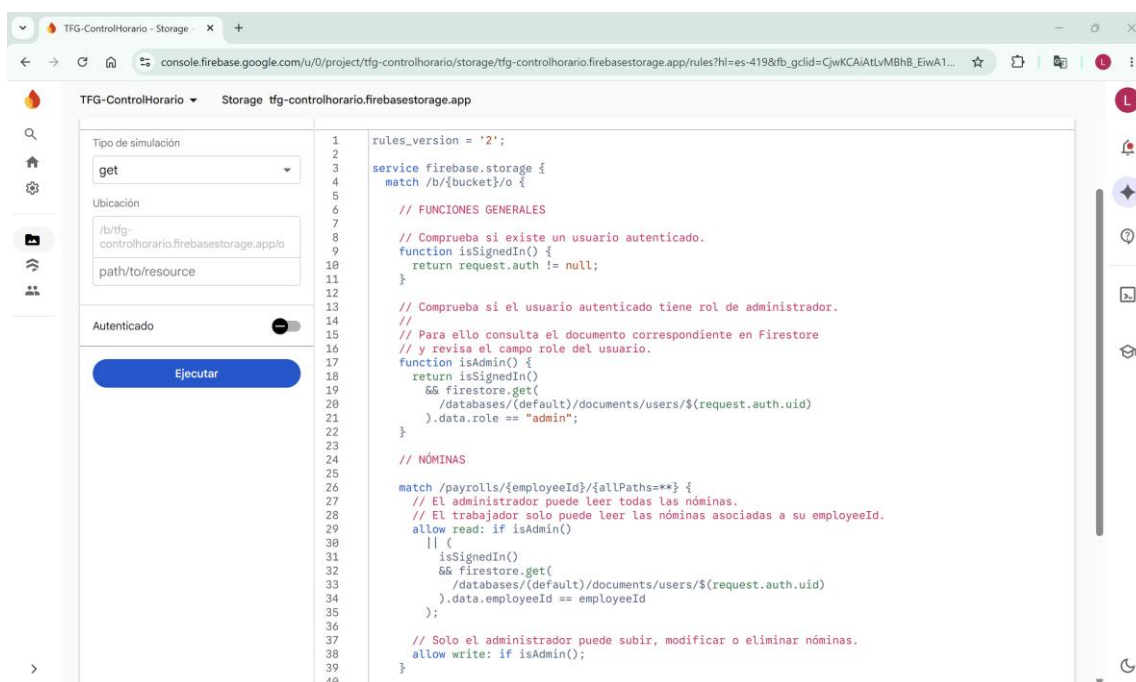


Figura 34. Reglas generales y acceso a nóminas.

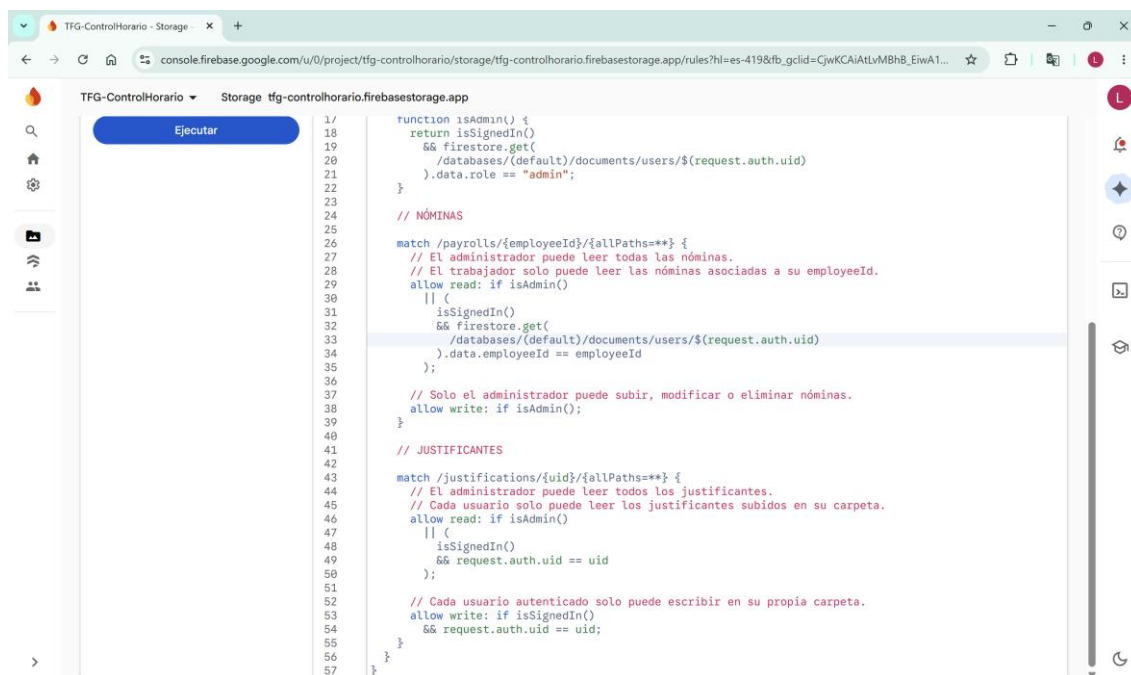


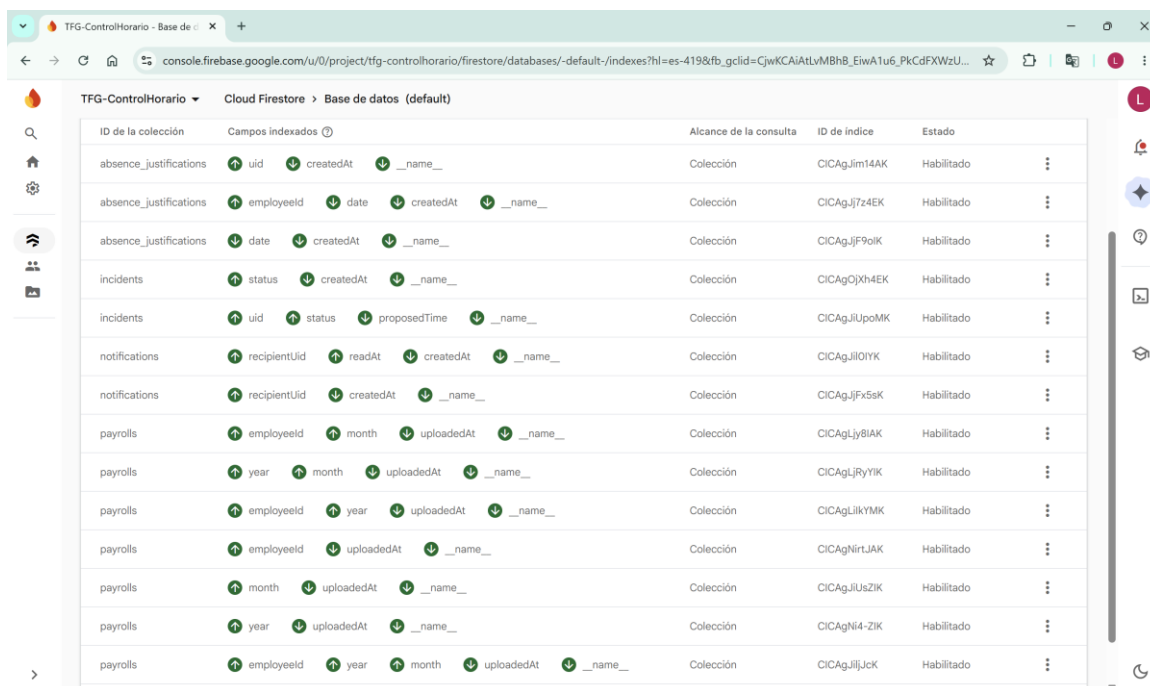
Figura 35. Reglas de acceso a nóminas y justificantes.

Este diseño asegura que el acceso a los archivos dependa estrictamente del usuario. Para las nóminas, el administrador cuenta con permisos totales, mientras que cada empleado solo puede consultar los documentos vinculados a su propia cuenta. En cuanto a los justificantes, los usuarios tienen restringida la lectura y escritura a su carpeta personal, impidiendo que un trabajador acceda a los archivos de otro.

Estas restricciones de almacenamiento funcionan en paralelo con las reglas de Cloud Firestore, blindando tanto los registros de la base de datos como los documentos subidos a la nube.

11.3 Índices de Cloud Firestore

El uso de consultas avanzadas con filtros múltiples ha requerido de la creación de varios índices compuestos en Cloud Firestore. Estos índices son imprescindibles para acelerar el rendimiento de las búsquedas y habilitar las consultas complejas en colecciones como incidencias, notificaciones, justificantes o nóminas. En la figura 36 se muestran estos índices, configurados para el correcto funcionamiento del sistema.



ID de la colección	Campos indexados	Alcance de la consulta	ID de índice	Estado
absence_justifications	uid, createdAt, __name__	Colección	CiAgJim14AK	Habilitado
absence_justifications	employeeid, date, createdAt, __name__	Colección	CiAgJ7z4EK	Habilitado
absence_justifications	date, createdAt, __name__	Colección	CiAgJf9oiK	Habilitado
incidents	status, createdAt, __name__	Colección	CiAgQjXh4EK	Habilitado
incidents	uid, status, proposedTime, __name__	Colección	CiAgJUpoMK	Habilitado
notifications	recipientId, readAt, createdAt, __name__	Colección	CiAgJlOIYK	Habilitado
notifications	recipientId, createdAt, __name__	Colección	CiAgJfX5sK	Habilitado
payrolls	employeeid, month, uploadedAt, __name__	Colección	CiAgLy8IAK	Habilitado
payrolls	year, month, uploadedAt, __name__	Colección	CiAgLjRyYK	Habilitado
payrolls	employeeid, year, uploadedAt, __name__	Colección	CiAgLlkYMK	Habilitado
payrolls	employeeid, uploadedAt, __name__	Colección	CiAgNirtJAK	Habilitado
payrolls	month, uploadedAt, __name__	Colección	CiAgJlJsZIK	Habilitado
payrolls	year, uploadedAt, __name__	Colección	CiAgNt4-ZIK	Habilitado
payrolls	employeeid, year, month, uploadedAt, __name__	Colección	CiAgJlJlJcK	Habilitado

Figura 36. Índices en Cloud Firestore.

Estas configuraciones se centran en las consultas que combinan filtros por usuario, estado o rangos de fechas con orden descendente por fecha de creación. Esta optimización es clave en módulos como nóminas, incidencias y notificaciones, donde la aplicación realiza búsquedas y filtrados en tiempo real que requieren una respuesta inmediata.

11.4 Recursos del proyecto

Para facilitar la revisión y evaluación del trabajo, a continuación, se detallan los accesos a los distintos componentes y materiales desarrollados en el proyecto.

11.4.1 Aplicación web desplegada

La aplicación está totalmente operativa y se puede probar directamente desde el navegador a través del siguiente enlace, sin necesidad de instalar nada: <https://tfg-controlhorario.web.app>

11.4.2 Repositorio de código fuente

Todo el código desarrollado para este Trabajo Fin de Grado está subido al siguiente repositorio público: <https://github.com/lmarvil1/TFG---Control-Horario>

En él se incluyen los archivos fuente de la aplicación, y el resto de recursos del desarrollo.

11.4.3 Versión para Android (APK)

El archivo instalable para dispositivos Android se ha subido al apartado de versiones (*Releases*) del propio repositorio: <https://github.com/lmarvil1/TFG---Control-Horario/releases>

11.4.4 Credenciales de demostración

Para poder entrar a la aplicación y probar algunos de los diferentes perfiles del sistema, se han habilitado estas dos cuentas de prueba con datos completamente ficticios:

- **Perfil Administrador**



- **Usuario:** admin.demo@gmail.com
- **Contraseña:** 123456
- **Perfil Empleado**
 - **Usuario:** empleado.demo@gmail.com
 - **Contraseña:** 123456

Al tratarse de cuentas de demostración vinculadas a correos ficticios, los mensajes de recuperación de contraseña que genera el sistema no llegarán a ninguna bandeja de entrada real. Sin embargo, utilizando las contraseñas indicadas arriba se puede acceder al instante y testear las funcionalidades de la aplicación sin restricciones.